AI ENGINEERING COURSE

Module 11

Agentic Engineering and Agent Frameworks

8 lessons · Personal offline study copy

Personal offline study copy. Content © Outcome School / Amit Shekhar. Not for redistribution. See study/NOTICE.md.

Contents

Module 11: Agentic Engineering and Agent Frameworks

- 11.1 What is Harness Engineering?
- 11.2 What is Loop Engineering?
- 11.3 What is Graph Engineering?
- 11.4 AI Is Only as Good as Our Definition of Done
- 11.5 How does LangChain work?
- 11.6 How does LangGraph work?
- 11.7 How does Claude Code work?
- 11.8 How does Cursor work?

MODULE 11

Agentic Engineering and Agent Frameworks

8 lessons · ~1 min overview

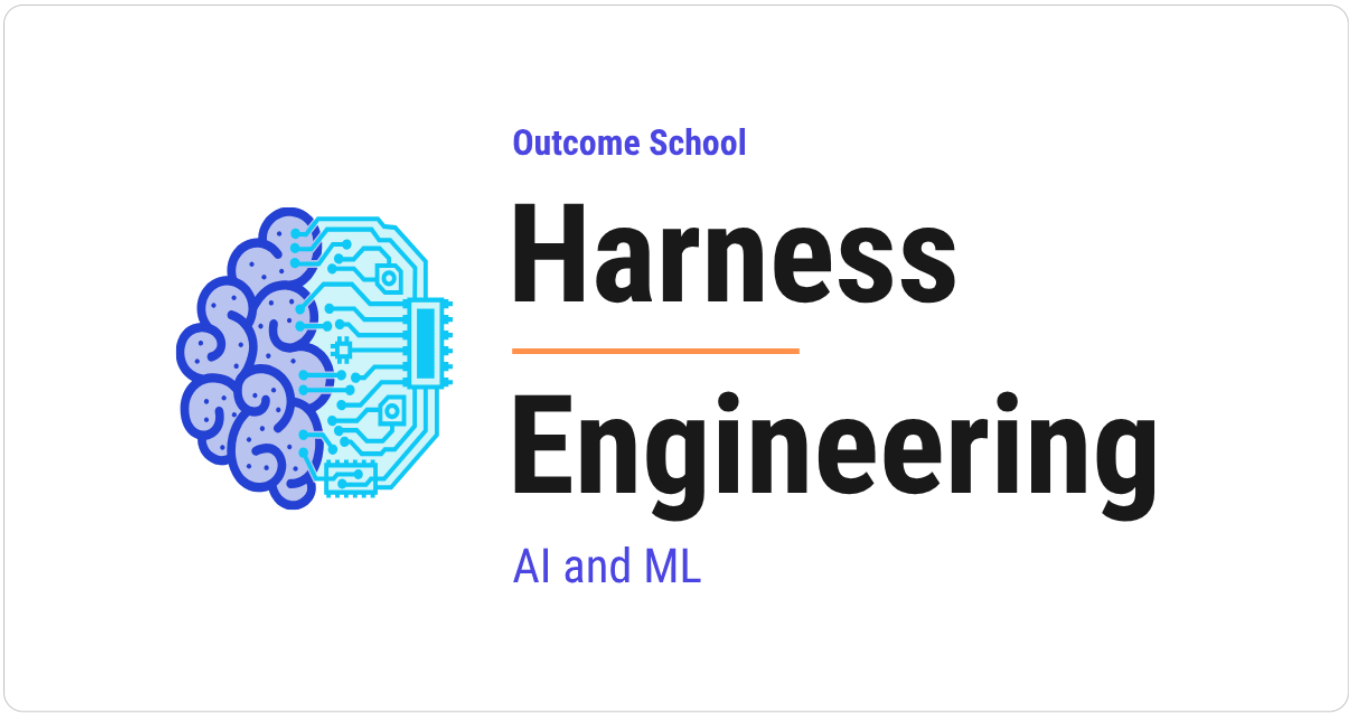
In this module, we will learn the engineering practices for building reliable agents, and then see how the popular frameworks and coding agents are built.

By the end of this module, we will know how to engineer the harness, the loop, and the graph around an agent, and how real coding agents work under the hood.

LESSON 11.1

What is Harness Engineering?

Lesson 11.1 · ~11 min · Published April 2, 2026 · https://outcomeschool.com/blog/harness-engineering-in-ai



Harness Engineering is everything we do to build the control layer around an AI model, which manages its inputs, outputs, tools, memory, errors, and evaluation, so that the model becomes actually usable in production.

In this blog, we will learn about Harness Engineering in AI. We will understand what a harness is, why we need it, and how it is used in AI Agents and evaluation systems.

We will cover the following:

- What is a Harness in AI?
- Why do we need Harness Engineering?
- Components of an AI Harness
- Harness Engineering for AI Agents
- Harness Engineering for Evaluation
- Best Practices in Harness Engineering
- Putting It All Together

Let's get started.

What is a Harness in AI?

Let's break the term:

Harness Engineering = Harness + Engineering

Harness means a control layer that helps you effectively use and manage a system..

Engineering means building it in a systematic and reliable way.

So, **Harness Engineering** is everything you do to build that control layer around the AI model to make it actually usable in production.

In simple words, the AI model alone is not enough. We need a layer of code around it that manages inputs, outputs, tools, memory, errors, and evaluation. This layer is the **harness**.

Just for the sake of understanding, let's say we have a very powerful engine. The engine alone cannot drive us anywhere. We need a car body, steering, brakes, fuel system, and dashboard around it. Together, they make the engine useful. The car body and all the parts around the engine - that is the harness.

Similarly, an AI model like an LLM is the engine. The harness is everything around it that makes the model useful in a real application.

Why Do We Need Harness Engineering?

An AI model by itself can only process the input we give and return the output. But in real-world applications, we need much more than that.

We want to use the model effectively so for that we need to:

- Use tools like search engines, databases, and APIs
- Remember past conversations
- Handle errors gracefully when something goes wrong
- Follow specific instructions and formats
- Evaluate for quality and correctness
- Deploy and monitor in production

Without a harness, the model is just a raw engine with no control. The harness gives us that control.

So, We need **Harness Engineering** for building this control layer around AI models.

Consider any AI-powered product we use today - a coding assistant, a chatbot, or an AI search engine. All of them have a harness around the model. The quality of the harness directly affects the quality of the product. **A great model with a poor harness will give a poor experience. A good model with a great harness will give an excellent experience.**

Components of an AI Harness

Now, let's understand the key components that make up an AI harness.

Prompt Management

This is the component that manages what goes into the model. It handles system prompts, user messages, templates, and context. It makes sure the model gets the right instructions every time. This is the discipline known as Context Engineering. Procedures that are too long to keep in the prompt at all times can live outside it as Agent Skills, which the harness loads only when a task matches.

Tool Orchestration

Many AI applications need the model to get assistant from external tools. For example, a coding assistant needs to read files, run commands, or search the web. The harness manages which tools are available, how they are called, and how the results are passed back to the model.

Memory Management

In a conversation, the model needs to remember what was said earlier. The harness manages this memory. It decides what to keep, what to remove, and how to compress old messages when the conversation gets too long.

Error Handling

Things can go wrong. The model can generate invalid output. A tool call can fail. The API can return an error. The harness handles all these cases so the application does not crash.

Input and Output Processing

The harness processes user input before sending it to the model. It also processes the model output before showing it to the user. This includes parsing, formatting, validation, and filtering.

Guardrails

These are safety checks built into the harness. They make sure the model does not generate harmful content, does not leak sensitive information, and stays within the boundaries of what it is supposed to do.

This was all about the key components of an AI harness. Now, let's move to the next section.

Harness Engineering for AI Agents

Now, let's understand how Harness Engineering is used for AI Agents.

An **AI Agent** is a system that uses tools, makes decisions, and takes multiple actions to complete complex tasks. It can read files, write code, search the internet, send messages, and more. The agent keeps working in a loop until the task is done.

The harness for an AI Agent is more complex because it needs to manage the entire agent loop. Designing that loop deliberately - the step budget, the observations, and the exit conditions - is Loop Engineering.

Here is how the agent harness works:

Step 1: The harness takes the user task and prepares the initial prompt with system instructions, available tools, and context.

Step 2: The harness sends this to the model and gets a response.

Step 3: The harness checks if the model recommends to use a tool. If yes, the harness executes the tool and sends the result back to the model.

Step 4: The harness repeats Steps 2 and 3 until the model says the task is complete.

Step 5: The harness presents the final result to the user.

This entire loop is managed by the harness. The model does the thinking, but the harness does the managing.

Here, we can see that without the harness, the agent cannot function. The harness is what turns a simple model into a powerful agent.

Claude Code is a real-world example of exactly this kind of agent harness. We have a detailed blog on how Claude Code works that walks through it step by step.

Note: The model itself does not execute any tool. It only decides which tool to call. The harness is the one that actually executes the tool and feeds the result back to the model.

Let's understand with a simple example:

```
User: "Find the weather in Delhi and send it to my email"
Harness Step 1: Prepare prompt with tools [weather_api, email_api]
Harness Step 2: Send to model
Model Response: "I will first check the weather. Call weather_api(city='Delhi')"
Harness Step 3: Execute weather_api -> Result: "32°C, Sunny"
Harness Step 4: Send result back to model
Model Response: "Now I will send the email. Call email_api(to='user@email.com', body='Weather in Delhi
Harness Step 5: Execute email_api -> Result: "Email sent"
Harness Step 6: Send result back to model
Model Response: "Done. I found the weather in Delhi (32°C, Sunny) and sent it to your email."
Harness Step 7: Present final response to user
```

Here, the model decided what to do, but the harness executed every action and managed the entire flow. This is how Harness Engineering works for AI Agents.

Harness Engineering for Evaluation

Now, let's understand how Harness Engineering is used for evaluating AI models.

An **evaluation harness** is a framework that runs a set of tests on an AI model and measures how well it performs.

For example, suppose we want to test if our model can answer math questions correctly. The evaluation harness will:

- Load a dataset of math questions with known correct answers
- Send each question to the model one by one
- Compare the model answer with the correct answer
- Calculate an overall score

This is similar to how we test software. We write test cases, run them, and check the results. The evaluation harness does the same thing for AI models.

The evaluation harness helps us answer important questions like:

- How accurate is the model?
- Did the model get better or worse after we made changes?
- How does one model compare to another?
- Where does the model fail?

Without an evaluation harness, we would have to test the model manually, which is slow and not reliable. The evaluation harness makes our life easy.

For open-ended outputs like chatbot replies where a single correct answer does not exist, the evaluation harness often uses LLM as a Judge to score quality where rule-based metrics fall short.

For AI Agents specifically, evaluation goes beyond the final answer - we must score the trajectory, the tool calls, and the plan as well. We have a detailed blog on AI Agent Evaluation that covers this in depth.

This was all about Harness Engineering for Evaluation. Now, let's look at some best practices.

Best Practices in Harness Engineering

Now, let's look at some best practices we must follow when building an AI harness.

Keep the harness modular. Each component like prompt management, tool orchestration, and memory management must be separate. This makes it easy to change one part without breaking others.

Log everything. Every input, output, tool call, and error must be logged. This helps us debug issues and understand what the model is doing.

Add guardrails from day one. Do not wait until something goes wrong. Build safety checks into the harness from the beginning.

Make tools reliable. If a tool fails, the harness must handle it gracefully. It can retry, use a fallback, or inform the model about the failure.

Test the harness, not just the model. The harness itself can have bugs. We must write tests for the harness code just like we test any other software.

Monitor in production. Once deployed, the harness must be monitored for latency, errors, cost, and quality. This helps us catch issues early. We have a detailed blog on AI Agent Observability that explains this in depth.

I will highly recommend following these practices from the start. It saves a lot of time in the long run.

Putting It All Together

Now, let's visualize how all the components of a harness work together:

```
User Input
    ↓
[Input Processing] → Clean and format the input
    ↓
[Prompt Management] → Build the prompt with context and instructions
    ↓
[AI Model (LLM)] → Generate a response
    ↓
[Output Processing] → Parse and validate the output
    ↓
[Tool Orchestration] → Execute any tool calls if needed
    ↓
[Memory Management] → Store the conversation for future reference
    ↓
[Guardrails] → Check for safety and compliance
    ↓
Final Output to User
```

Here, we can see that the AI model is just one part of the system. The harness is everything else. And in many real-world applications, the harness code is much larger than the model integration code.

This is how all the components of a harness work together to make an AI model useful in the real world.

As I keep saying: the model is the brain, but the harness is what makes the brain useful.

Frequently Asked Questions

Does the AI model execute tools by itself?

No. The model itself does not execute any tool. It only decides which tool to call. The harness is the one that actually executes the tool and feeds the result back to the model, and it keeps repeating this loop until the model says the task is complete.

Is a better model more important than a better harness?

Not always. The quality of the harness directly affects the quality of the product. A great model with a poor harness will give a poor experience, while a good model with a great harness will give an excellent experience. The model is the brain, but the harness is what makes the brain useful.

Is Harness Engineering the same as Context Engineering?

No. Context Engineering is one part of Harness Engineering. Prompt management, which handles system prompts, user messages, templates, and context, is the discipline known as Context Engineering. The harness also covers tool orchestration, memory management, error handling, input and output processing, and guardrails.

How does an evaluation harness score open-ended outputs?

For open-ended outputs like chatbot replies, where a single correct answer does not exist, the evaluation harness often uses LLM as a Judge to score quality. This is needed where rule-based metrics fall short. For AI Agents, evaluation also goes beyond the final answer to score the trajectory, the tool calls, and the plan.

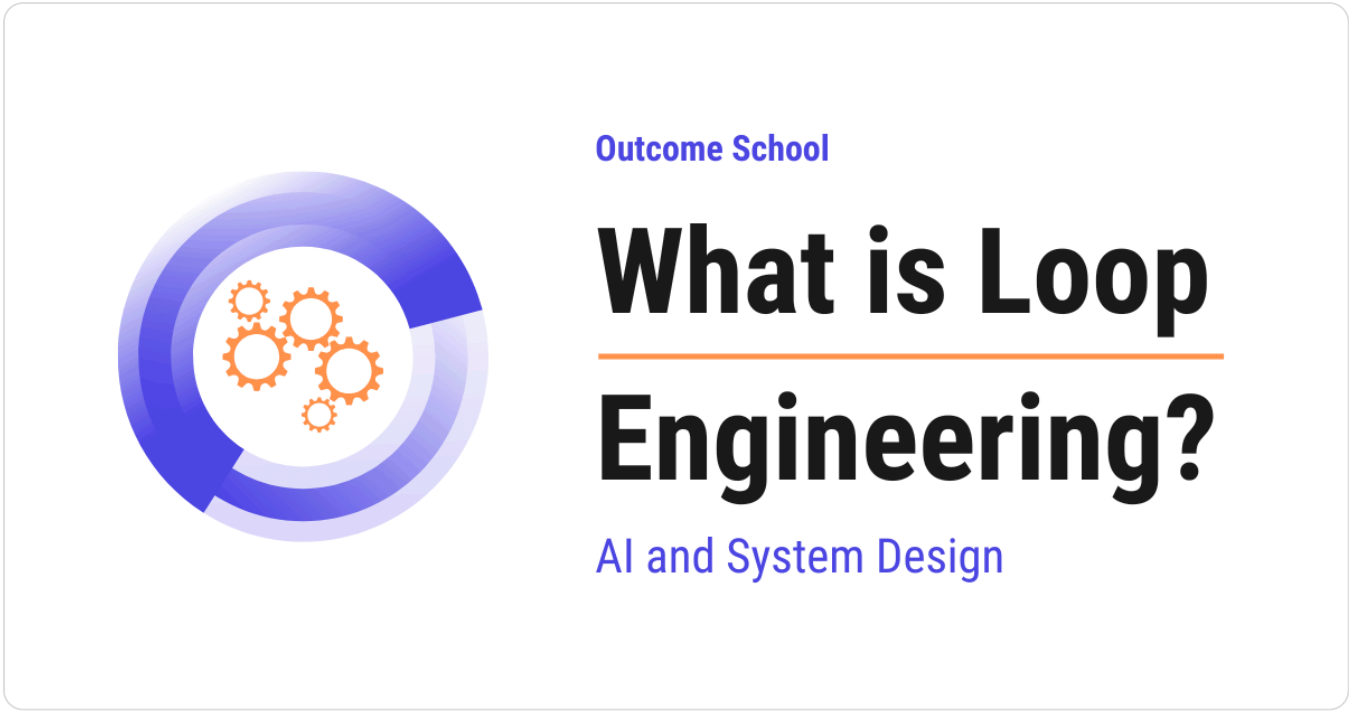
Do we need to test the harness code separately from the model?

Yes. The harness itself can have bugs, so we must write tests for the harness code just like we test any other software. In many real-world applications, the harness code is much larger than the model integration code.

LESSON 11.2

What is Loop Engineering?

Lesson 11.2 · ~20 min · Published August 20, 2026 · https://outcomeschool.com/blog/what-is-loop-engineering



Loop Engineering is the practice of designing the repeating cycle that an AI agent runs, so that the agent keeps making real progress on a task and stops at the right moment with the right result.

In this blog, we will learn about Loop Engineering, the practice of designing the repeating cycle that an AI agent runs again and again until a task is actually finished. We will also see why we need it, what one turn of the loop looks like, the parts of the loop that we must control, how it is different from prompt engineering and context engineering, the common ways a loop breaks, the techniques that fix those breaks, and where it works well and where it fails.

We will cover the following:

- What is Loop Engineering?
- Loop Engineering = Loop + Engineering
- Why do we need Loop Engineering?
- What is a loop in an AI agent?
- The simplest loop and its problems
- The parts of the loop that we must engineer
- Prompt Engineering vs Context Engineering vs Loop Engineering
- Common ways a loop breaks
- Techniques of Loop Engineering
- A complete example

- Where it works well and where it fails

Let's get started.

What is Loop Engineering?

Loop Engineering is the practice of designing the repeating cycle that an AI agent runs, so that the agent keeps making real progress on a task and stops at the right moment with the right result.

In simple words, a modern AI agent does not answer in one shot. It thinks, it does something, it looks at what happened, and then it thinks again. That circle repeats many times. Loop Engineering is the work of shaping that circle.

The model is only one part of an agent. The loop around the model decides how many times the model gets to try, what the model sees on each try, what it is allowed to do, and when the whole thing must stop. A strong model inside a badly designed loop will still fail. A modest model inside a well designed loop will often finish the job.

Loop Engineering = Loop + Engineering

Let's break the term into two parts.

Loop means something that repeats. The agent does a step, and then it does another step, and then another one, until some condition tells it to stop.

Engineering means designing that repetition on purpose, with rules, limits, and checks, instead of letting it run wild.

So, Loop Engineering is the deliberate design of the repeat.

The best way to learn this is by taking an example.

Let's say we are cooking a soup. We taste it. It needs salt. We add a pinch of salt. We taste it again. It is still not enough. We add another pinch. We taste again. Now it is good, so we stop and serve it.

Here, we can see a loop. Taste, decide, act, taste again.

Now think about what can go wrong. A cook who never tastes will never know when to stop. A cook who forgets that salt was already added will keep adding salt forever. A cook who tastes a hundred times will serve the soup cold. A cook who never decides that the soup is good enough will never serve it at all.

Loop Engineering is everything we do to prevent these problems. How many tastes are allowed, what the cook must remember, what "good enough" means, and what to do when the soup is already ruined.

An AI agent has exactly the same problems. Let me map the kitchen to the agent for your better understanding.

In the kitchen	In the agent
The cook	The AI model that decides what to do next
Adding a pinch of salt	An action, such as running a command or reading a file
Tasting the soup	Looking at the result of that action
Remembering the salt already added	The memory that carries from one turn to the next
"The soup is good enough"	The definition of done
"Serve it before it goes cold"	The limit on how many turns are allowed

Why do we need Loop Engineering?

Earlier, we used a language model like a vending machine. We put in a question, we got out an answer, and the story ended there. One request, one response.

But real tasks are not like that. Consider a task like "find the bug in this file and fix it". The model cannot do this in one shot, because it does not know what is inside the file yet. It must read the file first. After reading, it must run the test. After running the test, it must see the error. After seeing the error, it must edit the code. After editing, it must run the test again to confirm.

So, the work naturally becomes a sequence of steps, and each step depends on the result of the previous one. That is a loop.

Once we put a model inside a loop, a new class of problems arrives that never existed in the single-question world. The agent can repeat itself. It can forget what it already did. It can drift away from the original goal. It can run forever and burn a lot of money. It can declare success when nothing actually worked.

None of these are problems of the prompt. They are problems of the loop.

Now, let's take a real use-case. A coding agent that fixes a failing test, a research agent that reads twenty pages before writing a summary, and a support agent that checks an order, then checks the refund policy, then raises a ticket. Every one of them is a loop running behind the scenes. The company that builds the better loop ships the better agent, even when everyone is calling the very same model.

So, here comes Loop Engineering to the rescue.

What is a loop in an AI agent?

Before jumping into the loop, we must know one word that we will use again and again from here onwards.

The context is everything the model can see at this moment. It is the working notepad of the agent. The goal sits in it, the earlier steps sit in it, and every result sits in it. The model has no other memory. If a fact is not in the context, then for the model that fact does not exist.

Now, one turn of an agent loop has three parts.

Think: the model reads the context and decides what to do next.

Act: the system runs the thing the model asked for. The model cannot touch the outside world by itself, so it asks for a **tool**. A tool is simply a small program that we hand to the model, such as one that reads a file, searches the web, runs a command, or fetches data from another service.

Observe: the result of that action comes back and gets added to the context.

Then the turn ends, and the next turn begins with that updated context. Think, act, observe. Again and again.

This think, act, observe cycle is exactly the pattern behind a ReAct Agent, and we have a detailed blog on it that explains this in depth.

The important thing to notice is that the model itself does not loop. The model just produces one output when we call it. **The loop is code that we write around the model.** That is why we can engineer it.

> The model decides what to do next. The loop decides what actually happens.

The simplest loop and its problems

Let's see the most basic version of an agent loop as below:

```
while True:
    decision = model(context)
    result = run_tool(decision.tool, decision.input)
    context = context + result
```

Here, we can see that the model looks at the context, picks a tool, we run that tool, and we paste the result back into the context. Then it goes around again.

This works for a small demo, and it fails badly in the real world.

Problem 1: There is no stop. `while True` means the agent can go around a thousand times if the model never says it is finished.

Problem 2: The context only grows. Every raw tool result gets pasted in. After twenty steps, the context is huge, expensive, and full of noise.

Problem 3: There is no error handling. If the tool crashes, the whole loop crashes with it.

Problem 4: There is no definition of done. Nothing in this code knows what success looks like.

The issue with this approach is that it trusts the model to manage itself. Let's see how the next approach solve this issue.

The parts of the loop that we must engineer

There are seven parts in a loop, and each one is a decision that we, the engineers, must make.

The goal. The loop must carry a clear statement of what finished means. "Fix the bug" is weak. "The test named `test_login` passes" is strong, because a machine can check it.

The step budget. The loop must have a maximum number of turns. When the budget runs out, the agent stops and reports honestly that it did not finish.

The tools. The list of actions the agent is allowed to take. A short, sharp list of tools produces a much better loop than a long list of overlapping ones, because the model has fewer wrong turns available to it.

The observation. What we paste back after a tool runs. A tool that returns ten thousand lines of output will drown the loop. We must trim it, or summarize it, or return it in small pages, before it enters the context.

The memory. What carries forward from turn to turn. Full history is honest but expensive. A running summary is cheap but can lose details. This choice shapes everything.

The feedback on failure. When a tool returns an error, the message we paste back is a teaching signal. "Error" teaches nothing. "File not found at path `src/mian.py`, did you mean `src/main.py`" lets the agent recover on the very next turn.

The exit. The set of conditions under which the loop stops. Success, budget exhausted, repeated failure, or a request for a human. Every one of these must be handled.

Now, we have understood the parts. Let's see them inside real code.

Our updated loop:

```
MAX_STEPS = 12
def run_agent(goal, context):
    for step in range(MAX_STEPS):
        # the goal goes in on every single turn, so the agent cannot drift
        decision = model(goal, context)
        if decision.is_done:
            if verify(goal):          # check the claim, do not believe it
                return decision.answer
            context = add(context, "Verification failed. Keep working.")
            continue
        try:
            result = run_tool(decision.tool, decision.input)
        except ToolError as e:
            # a crash becomes a lesson instead of the end of the run
            result = f"Tool failed: {e}. Try a different approach."
        context = add(context, trim(result))
    return "Could not finish within the step budget."
```

Here, we have made six changes.

- `MAX_STEPS` gives the loop a hard ceiling, so it can never run forever.
- The `goal` is passed in on every turn, so the agent always knows where it is headed.
- `decision.is_done` gives the model a proper way to say that it has finished.
- `verify(goal)` checks that claim with real code instead of believing it. If the check fails, we tell the agent and let it continue.
- The `try` block turns a crash into a message, so a broken tool becomes a lesson instead of the end of the run.
- `trim(result)` keeps the observation small, so the context stays clean.

Note: the `verify` step is the single most valuable line here. Without it, an agent will happily tell us that the bug is fixed while the test is still failing.

This is how a loop stops being a demo and starts being a system.

Prompt Engineering vs Context Engineering vs Loop Engineering

These three names sound similar, so let's separate them clearly.

Prompt Engineering is about the wording of one instruction. What do we say to the model, and how do we say it.

Context Engineering is about the content of that notepad. What information sits in front of the model at this moment, where it came from, and what we left out.

Loop Engineering is about the shape of the repetition. How many turns, what changes between turns, what carries forward, and when it ends.

Let me tabulate the differences between the three for your better understanding.

Point	Prompt Engineering	Context Engineering	Loop Engineering
Unit of work	One instruction	One context	One full run
Main question	What do we say?	What does the model see?	What happens next, and when do we stop?
Typical fix	Rewrite the wording	Add, trim, or reorder information	Add a budget, a check, or a better exit
Failure it prevents	A vague or misread answer	A confused or overloaded model	An endless, drifting, or falsely successful run
Who controls it	The person writing the prompt	The code that gathers the information	The code that runs the model again and again

Prompt Engineering fixes what the model hears, Context Engineering fixes what the model knows, and Loop Engineering fixes what the model is allowed to do over time.

This was all about the comparison. Now, let's learn about the ways a loop breaks in practice.

Common ways a loop breaks

Most of the time, we do mistakes while designing the loop, and the same failures show up again and again.

The endless loop. The agent keeps taking turns and never decides that the work is complete. It usually happens when the goal has no definition of done that a machine can check.

The repeating loop. The agent runs the same failing command four times in a row and expects a different result. It happens when the failure message carries no new information, so the model has no reason to change its plan.

The forgetting loop. The agent solves a sub-problem on turn three, and by turn fifteen that solution has been squeezed out of the context. The agent then solves the same sub-problem again. This is a memory design failure.

The drifting loop. The agent starts by fixing a login bug and ends up rewriting the whole database layer. Each single step looked reasonable, and the destination is completely wrong. It happens when the original goal is not repeated in the context on every turn.

The lying loop. The agent announces that the task is complete, and nothing was actually verified. This is the most dangerous one, because the run looks successful in the logs.

The expensive loop. The agent finishes correctly, and it took ninety turns and a very large bill, because every raw tool output was pasted in and re-sent on every single turn.

Now that we have seen the failures, it is time to learn the techniques that prevent them.

Techniques of Loop Engineering

Give a step budget. Set a maximum number of turns and stop cleanly when it is reached. A run that reports "I could not finish in twelve steps, here is how far I got" is far more useful than a run that never returns.

Make done checkable by a machine. Do not ask the loop to decide that the code "looks correct". Ask it to run the test and read the pass or fail result that the computer itself reports. If a machine cannot check the goal, the loop cannot reliably end.

Verify before we trust. When the model says it is finished, run the check ourselves. If the check fails, put that fact into the context and continue the loop.

Repeat the goal every turn. Keep the original task at the top of the context on every single turn. This costs a few extra words on every turn, and it is the cheapest cure for drift.

Make failures informative. Turn an error into a hint. Instead of pasting the full crash report, paste the one line that matters plus a short note about what to try next.

Break the repeat. Detect when the same action is being attempted again with the same input. When that happens, block it and add a message such as "This exact command already failed twice. Try a different approach."

Trim every observation. Before a tool result enters the context, cut it down. The first few and the last few lines of a long output, the first twenty search results instead of all of them, the relevant function instead of the whole file.

Compact the history. When the context crosses a threshold, replace the old turns with a short summary that keeps the decisions and the facts learned, and drops the raw noise. The agent then continues with a light, clean context. We have a detailed blog on context compaction that explains this step by step.

Plan first, then loop. Ask for a short plan on the first turn, and keep that plan visible in the context. Each later turn then works against a written list instead of a vague memory, and the drift problem shrinks a lot. This is the idea behind the Plan-and-Execute Agent, and we have a detailed blog on it that covers the pattern end to end.

Use sub-loops. When a step is big, hand it to a separate agent with its own small loop, its own budget, and its own tools. The sub-agent returns only its final result. The parent loop stays short and clean, because all the noisy intermediate steps stayed inside the child.

Keep a human in the loop for irreversible steps. Deleting data, sending an email, moving money, deploying to production. The loop must pause and ask before crossing these lines.

Let me tabulate the failures against their fixes for your better understanding so that you know exactly which technique to reach for.

The failure	The fix
The endless loop	A step budget and a finish line a machine can check
The repeating loop	Better failure messages, and blocking a repeated action
The forgetting loop	Compact the history instead of dropping the old turns
The drifting loop	Repeat the goal every turn, and plan first
The lying loop	Verify the claim with code before exiting
The expensive loop	Trim every observation, and use sub-loops

A complete example

Let's take a real task. The goal is: "The test `test_login` must pass."

Turn 1. The agent reads the goal. It calls the tool that runs the test. The result comes back as a failure, and it points at line 42. We trim the output to the goal, the failing test name, and that one error line.

Turn 2. The agent reads line 42 and the function around it. It sees that a token is compared before it is created.

Turn 3. The agent edits the file to move the token creation above the comparison.

Turn 4. The agent runs the test again. It fails with a different error now, which tells us that the loop is making progress and not spinning.

Turn 5. The agent reads the new error, sees a missing import, and adds it.

Turn 6. The agent runs the test. It passes. The agent says it is done.

The verify step. The loop does not simply believe this. It runs the whole test suite one final time itself. Every test passes, so the loop exits and returns the answer.

Here, we can notice a few things. The loop had a budget of twelve turns and used six. The goal was checkable by a machine, so the ending was never in doubt. Each error message pointed at exactly one line, so every turn changed the plan. And the final claim was verified by code, not accepted on trust.

That is a well engineered loop.

Where it works well and where it fails

Where it works well:

- Tasks that have a clear finish line that a machine can check, such as tests passing, a build succeeding, or a file matching a required format.
- Tasks where the environment gives honest feedback after every action, such as a compiler, a test runner, or a service that returns a real error message.
- Tasks that can be broken into small steps, where a wrong step is cheap to undo.
- Long tasks where the agent must gather information before it can even plan.

Where it fails:

- Tasks with no objective finish line, such as "make this essay better". The loop has nothing to check, so it either stops too early or never stops.
- Environments that give slow or silent feedback. If a mistake shows up only thirty minutes later, the loop cannot learn from it in time.
- Actions that cannot be undone. A loop that experiments freely is a very bad fit for production systems and real money.
- Very long tasks where the context must hold hundreds of facts. The summarizing starts dropping things, and the forgetting loop appears.

Now we must have understood what Loop Engineering is. Prompt Engineering shapes one sentence, Context Engineering shapes what the model sees, Graph Engineering shapes the path from one step

to the next, and Loop Engineering shapes the whole run from the first turn to the last. As agents take on longer and longer work, the quality of the loop, and not the cleverness of the prompt, decides whether the work actually gets finished.

Frequently Asked Questions

Can a stronger model make up for a badly designed loop?

No. A strong model inside a badly designed loop will still fail, while a modest model inside a well designed loop will often finish the job. The loop decides how many times the model tries, what it sees on each try, what it is allowed to do, and when it must stop.

Should we trust an agent when it says the task is done?

No. We must verify the claim with real code before the loop exits, such as running the tests ourselves. If the check fails, we put that fact into the context and let the agent continue. Without this step, an agent can report that a bug is fixed while the test is still failing.

What should an agent do when it runs out of its step budget?

It should stop cleanly and report honestly that it did not finish. A run that says it could not finish within the budget and shows how far it got is far more useful than a run that never returns. The step budget is what keeps the loop from running forever.

Is it good to paste the full tool output back into the context?

No. If every raw tool result is pasted in, the context keeps growing and becomes huge, expensive, and full of noise. We must trim each observation first, such as keeping only the relevant lines, the first few search results, or the one function we need instead of the whole file.

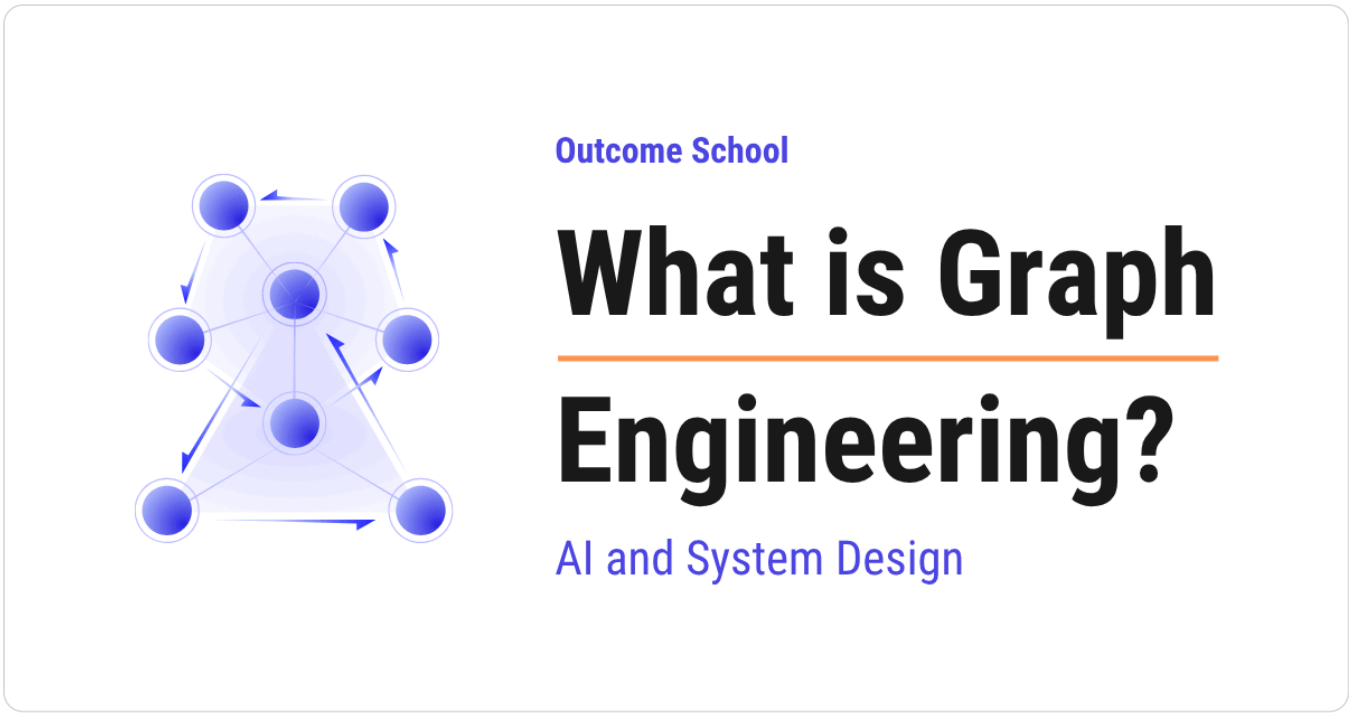
What is the difference between Loop Engineering and Graph Engineering?

Graph Engineering shapes the path from one step to the next, while Loop Engineering shapes the whole run from the first turn to the last. Loop Engineering focuses on the repeating think, act, observe cycle of an agent, including its step budget, memory, feedback on failure, and exit conditions.

LESSON 11.3

What is Graph Engineering?

Lesson 11.3 · ~24 min · Published August 21, 2026 · https://outcomeschool.com/blog/what-is-graph-engineering



Graph Engineering is the practice of designing an AI system as a graph, where every step of the work is a node, and every path from one step to another is an edge. We break the work into small steps and clearly decide which step runs after which.

In this blog, we will learn about Graph Engineering, the practice of building an AI system as a graph of small steps connected by clear paths instead of one giant prompt or one endless loop. We will also see why we need it, what nodes and edges actually mean, how the state travels through the graph, how conditional edges take decisions, how cycles let the system do the work again, how parallel branches save time, how checkpoints let us pause and resume, and where it works well and where it fails.

We will cover the following:

- What is Graph Engineering?
- Graph = Nodes + Edges
- Why do we need Graph Engineering?
- The three building blocks: Node, Edge, and State
- Let's build our first graph
- Conditional edges: taking decisions inside the graph
- Cycles: doing the work again when needed
- One full run, step by step
- Parallel branches: doing many things at the same time

- Checkpoints: pause and resume the graph
- Human in the loop
- Handling errors inside a graph
- Graph Engineering vs Loop Engineering
- Where Graph Engineering works well
- Where Graph Engineering fails
- Best practices in Graph Engineering
- Conclusion

Let's get started.

What is Graph Engineering?

Graph Engineering is the practice of designing an AI system as a graph, where every step of the work is a node and every path from one step to another step is an edge.

Graph Engineering = Graph + Engineering.

In simple words, instead of writing one huge instruction and hoping that the model does everything correctly in one go, we break the work into small steps, and then we clearly decide which step runs after which step.

Here, whenever we say the model, we mean the AI model that reads text and writes text, and whenever we say a tool, we mean anything outside the model that our system can call, like a search, a database, or an email service.

We hear this word mostly around AI agents today, and there is a good reason for that. Real work is never one step. Real work is many steps, with going back, waiting, checking, and approving in between. Graph Engineering is how we give that shape to our system.

Graph = Nodes + Edges

Before jumping into Graph Engineering, we must know what a graph is.

A graph is just two things.

- **Nodes**: the places where something happens.
- **Edges**: the paths that connect one place to another place.

The best way to learn this is by taking an example.

Let's say we are looking at the metro map of a city. Every station on that map is a node. Every track between two stations is an edge. A train starts from one station, travels on a track, reaches the next

station, and continues like this until it reaches the last station.

Now, replace the stations with the steps of our work, and replace the tracks with the order of our work. That is exactly the graph we build in Graph Engineering.

Metro map	Our AI system
Station	Node, which means one small step of work
Track between two stations	Edge, which means the path to the next step
Bag that the passenger carries	State, which means the information that travels along
First station	The step where our system starts
Last station	The step where our system stops

Here, we can see that an edge is simply the answer to one small question. After this step is over, which step runs next?

Now, we have understood the graph. Now, let's understand why we need Graph Engineering at all.

Why do we need Graph Engineering?

Let's say we are building a system that answers questions about a company's internal documents. A user asks a question, and our system must give a correct answer with proper sources.

We will build this system step by step. At every step we will see the problem, and then we will move to a better approach.

Approach 1: One big prompt

We write one long instruction and send it to the model.

```
You are a helpful assistant. Read the user question. Search the company
documents. Read the results. Write an answer. Check that the answer is
correct. Add the sources. If something is missing, search again.
```

The model reads all of this and tries to do everything at once.

Problem: The model does some parts and quietly forgets the other parts. Sometimes it writes the answer without searching. Sometimes it searches but never checks the answer. And when the output is wrong, we have no idea which part went wrong, because everything happened inside one single call. We cannot fix what we cannot see.

The issue with this approach is that there is no structure at all. Let's see how the next approach solve this issue.

Approach 2: A chain of steps

Now, we break the work into separate calls, and we run them one after another.

```
Step 1: understand the question
Step 2: search the documents
Step 3: write the answer
Step 4: send the answer
```

This is prompt chaining, and it is much better. Every step has only one job. When something goes wrong, we know exactly which step failed.

Problem: A chain always moves forward in a straight line. It never comes back. But real work needs coming back. What if the search returns nothing useful? What if the answer is wrong and needs to be written again? A straight chain cannot do this. It simply walks forward and delivers a bad answer.

The issue with this approach is that it cannot go back. Let's see how the next approach solve this issue.

Approach 3: One loop

Now, we put the whole thing inside an agent loop. The model thinks, calls a tool, looks at the result, and thinks again, again and again, until it decides that the work is done.

This solves the coming back problem. The model can search again, write again, and correct itself.

Problem: Now we have gone to the other extreme. Every decision is taken by the model inside one loop, so we have very less control. The model repeats the same failing search ten times. It skips the checking step because it feels confident. It calls a dangerous tool that we never wanted it to call at that moment. The path is different on every run, so testing becomes very hard.

The issue with this approach is that the flow is completely in the model's hands. Let's see how the next approach solve this issue.

Approach 4: A graph

So, here comes Graph Engineering to the rescue.

We keep the small steps of Approach 2, we keep the coming back ability of Approach 3, and we write down the paths ourselves.

```
understand  ->  search  ->  write  ->  check
check       ->  send        (when the answer is good)
check       ->  search      (when the sources are missing)
check       ->  write       (when the answer is weak)
```

Here, we can see that every step is small and visible, the system can go back when it needs to, and the model takes decisions only at the places where we allow it to take decisions.

The problem is solved. This is Graph Engineering.

The example we took here, where the system searches, writes, checks, and searches again, is what we call Agentic RAG, and we have a detailed blog on it that explains this in depth.

The three building blocks: Node, Edge, and State

Every graph we build has only three parts.

Node: A node is one step of the work. It is just a function. It takes the current information, does one small job, and gives the updated information back. A node can call a model, call a tool, hit a database, or run plain code with no model at all.

Edge: An edge is a path from one node to another node. It answers only one question. Which node runs next?

State: The state is the shared notebook of the graph. Every node reads from this notebook and writes into this notebook. This is how the second node knows what the first node did.

Let's see the state for our document question system as below:

```
state = {
    "question": "What is our refund policy?",
    "intent": None,
    "documents": [],
    "answer": None,
    "score": None,
    "attempts": 0,
}
```

Here, we have created a simple notebook with empty values. As the graph runs, the nodes will keep filling these values. The `understand` node fills `intent`. The `search` node fills `documents`. The `write` node fills `answer`. The `check` node fills `score` and `attempts`.

This state is the most important design decision in Graph Engineering. If the state is clean, the graph stays simple. If the state becomes a dumping ground of random keys, the graph becomes impossible to understand. Whatever a node puts in front of the model also comes out of this state, so keeping it clean is good Context Engineering as well.

Now that we have learned about the building blocks, it is time to build a real graph.

Let's build our first graph

First, we will write the nodes. Every node is a plain function.

```
def understand(state):
    state["intent"] = model.find_intent(state["question"])
    return state
def search(state):
    state["documents"] = db.search(state["intent"])
    return state
def write(state):
    state["answer"] = model.write_answer(state["question"], state["documents"])
    return state
```

Here, we can notice that each function does exactly one job and returns the updated state. There is no big instruction anywhere. The `understand` node only finds the intent. The `search` node only searches. The `write` node only writes the answer from the documents that the `search` node found.

Then, we will connect these nodes with edges.

```
graph = Graph(state)
graph.add_node("understand", understand)
graph.add_node("search", search)
graph.add_node("write", write)
graph.add_edge("understand", "search")
graph.add_edge("search", "write")
graph.set_entry("understand")
graph.set_finish("write")
```

Here, we have first added the three nodes to the graph. Then, we have added two edges. The first edge says that after `understand`, the `search` node runs. The second edge says that after `search`, the `write` node runs. Finally, we have told the graph where to start and where to stop.

After this, we can run the graph.

```
final_state = graph.run(state)
print(final_state["answer"])
```

Here, we are giving the starting notebook to the graph. The graph walks from node to node, each node updates the notebook, and at the end we read the answer from the same notebook.

It works perfectly. But this graph is still a straight line, exactly like Approach 2. Now, let's give it the power to take decisions.

Note: The code in this blog is kept simple just for the sake of understanding. The real libraries built for this work give us the same three ideas of node, edge, and state, only with slightly different names. LangGraph is the most popular one of them, and we have a detailed blog on it that explains how it works step by step.

Conditional edges: taking decisions inside the graph

A normal edge always goes to the same next node. A **conditional edge** looks at the state first and then decides which node runs next.

Let's add a `check` node that gives a score to our answer.

```
def check(state):
    state["score"] = model.rate_answer(state["answer"], state["documents"])
    state["attempts"] = state["attempts"] + 1
    return state
```

Here, we have used the model as a reviewer, which is the LLM as a Judge pattern. It reads the answer along with the documents, and it gives a score out of ten. We are also counting the attempts, and we will see very soon why this count matters so much.

Now, we will write the routing function.

```
def route_after_check(state):
    if state["score"] >= 8:
        return "send"
    if not state["documents"]:
        return "search"
    return "write"
```

Here, we can see that the decision is plain code, and it is fully in our hands. If the score is good, we go to `send`. If there were no documents at all, we go back to `search`. In every other case, we go back to `write` and try again.

Then, we attach this function as a conditional edge.

```
graph.add_node("check", check)
graph.add_node("send", send)
graph.add_edge("write", "check")
graph.add_conditional_edge("check", route_after_check)
graph.set_finish("send")
```

We have followed the following steps:

- We have added the `check` node and the `send` node. The `send` node is a small node that simply delivers the final answer to the user.
- We have added a normal edge from `write` to `check`, so every answer goes for review.
- We have added a conditional edge from `check`. This edge has three possible destinations, and our routing function picks one of them by reading the state.
- We have moved the finish point from `write` to `send`, because `send` is the last station of our graph now.

Note: The model gave the score, but the model did not choose the path. The path was chosen by our own code. This is the heart of Graph Engineering. We let the model do the thinking, and we keep the control of the flow with ourselves.

Cycles: doing the work again when needed

In the graph above, `check` can send us back to `write`, and `write` sends us to `check` again. This going back and forth between two nodes is called a **cycle**.

A cycle is a very powerful thing, because this is how a system corrects itself. Write, review, write better, review again. This is exactly how a human works, and this is also the idea behind a Reflection Agent.

But, here is the catch. A cycle can run forever. If the reviewer never gives a score of eight, our graph will keep writing and checking until our money is over.

So, every cycle must have a stop condition. This is why we were counting the attempts inside the `check` node.

```
def route_after_check(state):
    if state["score"] >= 8:
        return "send"
    if state["attempts"] >= 3:
        return "ask_human"
    if not state["documents"]:
        return "search"
    return "write"
```

Here, we have added two highlighted lines. After three attempts, the graph stops trying on its own and hands the work over to a human through a new `ask_human` node. We must also add this node to the graph, otherwise the graph will not know where this path goes.

```
graph.add_node("ask_human", ask_human)
graph.add_edge("ask_human", "send")
```

Here, we have added the `ask_human` node and joined it to `send`, so even this sad path reaches the last station properly.

Our full graph looks like below now:

```
understand  ->  search  ->  write  ->  check
check       ->  send        (the score is good)
check       ->  ask_human   (three attempts are over)
check       ->  search      (there are no documents)
check       ->  write       (the answer is weak)
ask_human   ->  send
```

Here, we can see the complete shape of our system on one screen. Every step is a node, every line is an edge, and there is no hidden behaviour anywhere.

Our system now fails in a safe and predictable way instead of running forever.

Very important: Every cycle in our graph must have a hard limit.

One full run, step by step

Now, let's walk through one complete run and see what the notebook looks like after every node. This is the best way to feel how a graph really works.

Step 1: The user asks a question, so the starting state is:

```
question: "What is our refund policy?"
intent: None, documents: [], answer: None, score: None, attempts: 0
```

Step 2: The `understand` node runs and fills the intent:

```
intent: "refund policy duration and conditions"
```

Step 3: The `search` node runs and fills the documents:

```
documents: ["policy-v3.pdf", "faq-refunds.md"]
```

Step 4: The `write` node runs and fills the answer:

```
answer: "Refunds are allowed within 30 days."
```

Step 5: The `check` node runs. The answer has no sources in it, so the reviewer gives a low score:

```
score: 5, attempts: 1
```

Step 6: The conditional edge reads the state. The score is below eight, the documents are present, and the attempts are below three, so the path goes back to `write`.

Step 7: The `write` node runs again, and this time it writes a better answer with sources:

```
answer: "Refunds are allowed within 30 days of purchase, as per policy-v3.pdf."
```

Step 8: The `check` node runs again:

```
score: 9, attempts: 2
```

Step 9: The conditional edge reads the state again. The score is eight or more, so the path goes to `send`, and the graph stops there.

Here, we can notice one beautiful thing. The system found its own mistake and corrected it, and still it never went out of our control, because we had written every path ourselves.

This was all about one single run of our graph. Now, let's see how we can make our graph faster.

Parallel branches: doing many things at the same time

Sometimes two steps have nothing to do with each other. Searching our internal documents and searching the public web are completely independent. Running them one after another only wastes time.

In a graph, we can send one node to two nodes at the same time.

```
graph.add_edge("understand", "search_docs")
graph.add_edge("understand", "search_web")
graph.add_edge("search_docs", "merge")
graph.add_edge("search_web", "merge")
```

Here, we can see two things. First, `understand` has two outgoing edges, so both the searches start together. This is called fan-out. Second, both the searches point to the same `merge` node, so the graph waits for both of them to finish and then continues. This is called fan-in.

The `merge` node joins the two results.

```
def merge(state):
    state["documents"] = state["doc_results"] + state["web_results"]
    return state
```

Here, we have simply joined both the lists into the `documents` key, and the rest of our graph does not even need to know that two searches happened.

Fan-out and fan-in are two of the common patterns that we use to coordinate many steps together. We have a detailed blog on AI Orchestration that covers these patterns end to end.

Note: When two nodes run at the same time, they must write into different keys of the state. If both write into the same key, one will silently overwrite the other, and we will spend hours finding that bug.

Checkpoints: pause and resume the graph

A long graph takes time. It calls models, it calls tools, and it waits for slow services. Now, what happens if our server restarts in the middle of a run? In a simple loop, everything is lost, and the user

has to start again from zero.

Here comes **checkpointing** into the picture.

After every node finishes, the graph saves the state. This saved state is called a checkpoint. If something breaks, we load the last checkpoint and continue from that exact node.

```
graph = Graph(state, checkpointer=Database())
final_state = graph.run(state, thread_id="user-123")
```

Here, we have given a storage to the graph, and we have given an id to this particular run. After every node, the state is saved against this id. If this run breaks after the `search` node, we call `run` again with the same `thread_id`, and the graph starts from `write` instead of starting from `understand`.

This is possible only because the whole progress of our system lives inside one plain state object. This is one more reason why keeping a clean state is so valuable.

Till now, we have learned how a graph runs on its own. Now, it's time to learn about the place where a human joins the graph.

Human in the loop

Some steps must never happen without a human saying yes. Sending an email to a customer. Issuing a refund. Deleting a record. Pushing code to the live app.

Because our work is divided into nodes, we can simply stop the graph before such a node.

```
graph.add_node("send_email", send_email, interrupt_before=True)
```

Here, we have marked one node as a stopping point. When the graph reaches `send_email`, it saves the state and pauses. Nothing is sent yet.

Now, a human opens the pending work, reads the draft, and takes a call.

```
if human_says_yes:
    graph.resume(thread_id="user-123")
else:
    graph.update_state(thread_id="user-123", changes={"answer": edited_answer})
    graph.resume(thread_id="user-123")
```

Here, we can see that the human can either allow the graph to continue, or first correct the answer inside the state and then allow it to continue. The graph does not even know that a human touched it. It just reads the state and moves ahead.

This is how we get the speed of automation with the safety of human review.

Handling errors inside a graph

Tools fail. Networks fail. Models return broken output. In one big prompt, a failure kills everything. In a graph, a failure is just one node behaving badly, and we can keep it inside the state like any other information.

```
def search(state):
    try:
        state["documents"] = db.search(state["intent"])
        state["error"] = None
    except Exception:
        state["error"] = "search service is down"
    return state
```

Here, we have caught the failure and written it into the state instead of letting the whole graph crash.

Then, we write a routing function that reads that error and decides what to do.

```
def route_after_search(state):
    if state["error"]:
        return "search_web"
    if not state["documents"]:
        return "ask_human"
    return "write"
```

Then, we replace the normal edge coming out of `search` with a conditional edge as below:

```
graph.add_conditional_edge("search", route_after_search)
```

Here, we can notice that a failure is now just a normal path in our graph. When our own search is down, we fall back to the web search. When there is nothing at all to answer from, we ask a human. Nothing crashes, and the user always gets some response.

This is how we build a system that keeps working even on a bad day.

Now that we have learned about the complete graph, it's time to compare it with the other popular way of building AI systems.

Graph Engineering vs Loop Engineering

Loop Engineering is about one agent thinking in a loop, deciding its own next action, and stopping when it feels that the work is done. Graph Engineering is about many small steps, with the paths between them written down by us.

Let me tabulate the differences between Loop Engineering and Graph Engineering for your better understanding.

Point	Loop Engineering	Graph Engineering
Who decides the next step	The model decides	Our code decides, using the model's output
Structure	One loop of think and act	Many nodes joined by edges
Path on every run	Different every time	Same path for the same situation
Testing	Hard, because the path keeps changing	Easy, we can test one node at a time
Finding a bug	We read one long history of everything the model did	We look at the one node that failed
Freedom for the model	Very high	Only where we allow it
Best for	Open ended and unknown tasks	Known work with clear stages

---

Where Graph Engineering works well

- **Work with clear stages.** Support tickets, refund handling, document review, and report writing all have a natural shape of stages.
- **Work that needs approval.** Because we can pause at any node, human review fits in very naturally.
- **Work that must be repeatable.** The same input takes the same path, so we can test it and trust it.
- **Long running work.** Checkpoints let a graph survive restarts and continue for hours or even days.
- **Work with independent parts.** Parallel branches finish many searches or many checks together.
- **Work that must be explained.** We can show the exact path that was taken, node by node. In banking, healthcare, and legal work, this matters a lot.

Where Graph Engineering fails

- **Truly open ended work.** If we cannot even name the stages, then drawing a graph is only guesswork. A loop is a better fit there.
- **Very simple work.** If one model call answers the question, a graph only adds work with no benefit.
- **Graphs that grow without care.** A graph with sixty nodes and edges going everywhere is worse than a big prompt, because now the mess is spread across many files.
- **A messy state.** When every node adds a few keys and nobody removes anything, the state becomes a junkyard, and no node can be understood on its own.
- **Hidden dependencies.** If a node silently needs a key that only some earlier node writes, then moving the nodes around will break the graph in a way that is very hard to find.

- **Cycles without limits.** One missing stop condition, and our graph keeps running in circles all night, and we pay for every round.

Best practices in Graph Engineering

Keep every node small. One node, one job. If we cannot describe a node in one short sentence, then it must be split into two nodes.

Design the state first. Write down every key, what it holds, and which node fills it. Do this before writing a single node.

Keep the decisions in code, not in prose. Let the model give the score, the label, or the choice. Let our own code read that value and pick the path. This one habit gives us most of the control.

Put a limit on every cycle. Count the attempts inside the state, and always have an exit path for the case when the count is over.

Always have an exit. Every path in the graph must reach an end, even the sad paths. A path that goes nowhere is a hung system.

Turn failures into paths. Write the error into the state, and route on it. Do not let one bad tool call kill the whole run.

Draw the graph on paper. If we cannot draw it, then our teammates and our users will never understand it. A graph that cannot be drawn is too complex.

Log every node. Save which node ran, what came in, and what went out. When something goes wrong, this log takes us straight to the guilty node. This is what gives us real observability into our system.

Test node by node. Every node is just a function from state to state, so we can test it alone with a fixed state and no model at all. This is a huge advantage over one big prompt.

Start with a straight line. Build the simplest working path first. Add branches, cycles, and parallel work only when a real problem asks for them.

Conclusion

Graph Engineering is a simple idea. Break the work into small steps, write down the paths between those steps, and keep all the progress inside one shared state.

When we do this, our AI system stops being a black box that sometimes works. It becomes a system that we can see, test, pause, resume, correct, and explain to anyone.

The model still does the thinking. The graph decides where that thinking goes.

Now we must have understood Graph Engineering, and we can start drawing our own graph for the work that we are building.

Frequently Asked Questions

Is Graph Engineering the same as prompt chaining?

No. Prompt chaining breaks the work into separate calls that run one after another, but a chain always moves forward in a straight line and never comes back. Graph Engineering keeps those small steps and adds the ability to go back, branch, and take decisions, with the paths between steps written down by us.

Does the model choose the next step in Graph Engineering?

No. The model can give a score, a label, or a choice, but our own code reads that value from the state and picks the path. We let the model do the thinking, and we keep the control of the flow with ourselves. This is the heart of Graph Engineering.

Does every node in a graph need to call an AI model?

No. A node is just a function that takes the current state, does one small job, and returns the updated state. A node can call a model, call a tool, hit a database, or run plain code with no model at all. This is also why we can test each node alone.

Is LangGraph the same thing as Graph Engineering?

No. Graph Engineering is the practice of designing an AI system as nodes, edges, and state. LangGraph is the most popular library built for this work. Such libraries give us the same three ideas of node, edge, and state, only with slightly different names.

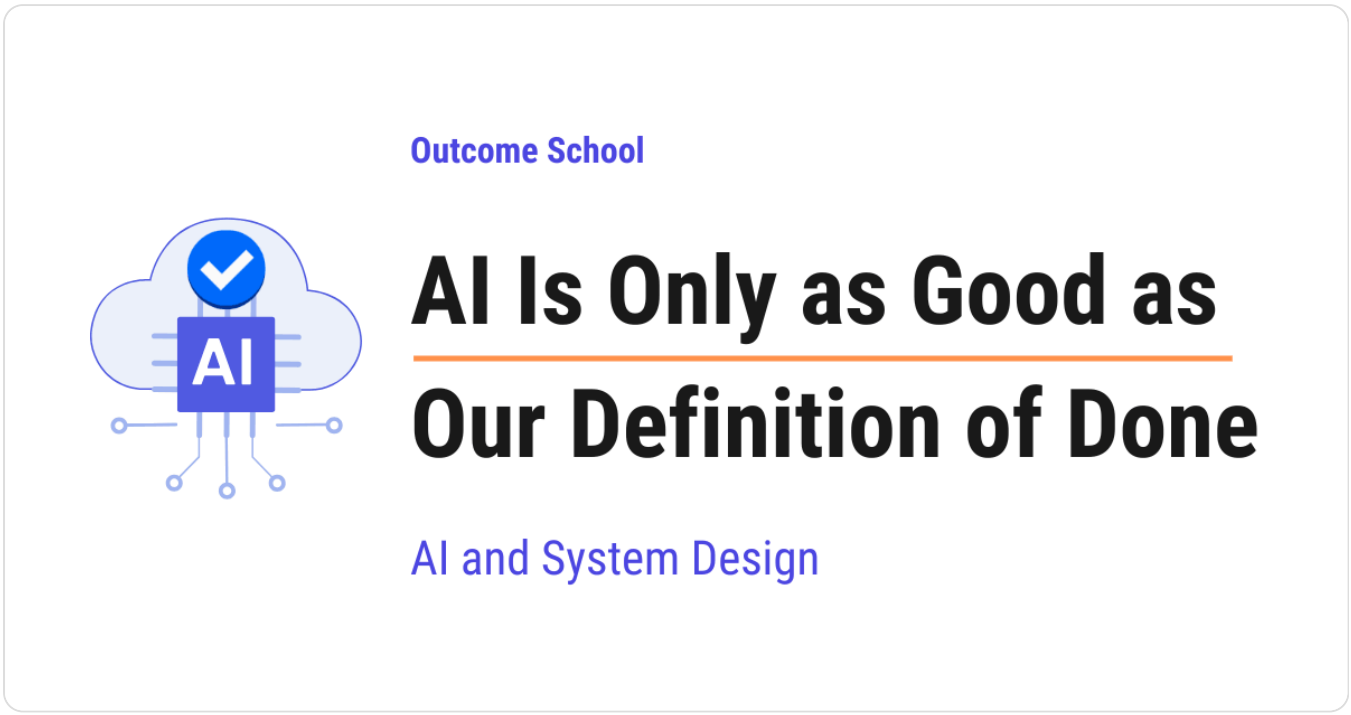
What happens if two parallel nodes write to the same state key?

One will silently overwrite the other. When two nodes run at the same time in parallel branches, they must write into different keys of the state. A merge node can then join the results, so the rest of the graph does not need to know that two branches ran together.

LESSON 11.4

AI Is Only as Good as Our Definition of Done

Lesson 11.4 · ~10 min · Published August 25, 2026 · https://outcomeschool.com/blog/ai-is-only-as-good-as-our-definition-of-done



AI does its best work when we can clearly check whether a task is finished, and it struggles when only a human can judge the result. That clear check is our definition of done.

In this blog, we will learn about one simple idea, AI is only as good as our definition of done, which means the clear line we draw between a task that is finished and a task that is not. We will also see what a definition of done really is, why AI feels magical on the tasks that a machine can check, why it feels unreliable on the tasks that only a human can judge, how the loop that AI runs behind the scenes creates this exact gap, and how we can write a definition of done that turns a vague task into a checkable one.

We will cover the following:

- What is a definition of done?
- Tasks where the definition of done is exact
- Tasks where the definition of done is fuzzy
- Why AI is so strong exactly where we can measure
- How to write a better definition of done

Let's get started.

What is a definition of done?

Whenever we give a task to AI, two things are involved.

The first thing is the goal. What do we want?

The second thing is the checker. How do we find out whether the AI actually did it?

Definition of Done = Goal + A way to check the goal

Most of the time, we give the goal and forget the checker. We say "build this feature", the AI gives us two hundred lines of code, and then we sit and read those lines to decide whether it is correct. In that case, we ourselves are the checker.

A checker, also called a verifier, is anything that can look at the output and say pass or fail without any opinion. A unit test is a checker. A compiler is a checker. A human reading the code is also a checker, but a slow one that gives a different answer on a tired day.

Here is the point of this blog. AI is extremely strong on the tasks that have a fast and exact checker, and unreliable on the tasks that do not have one. The difference is not in the intelligence of the model. It is in our definition of done.

Tasks where the definition of done is exact

The best way to learn this is by taking an example.

Suppose we write the test first, like below:

```
@Test
fun `should return the discounted price`() {
    val price = calculateDiscountedPrice(1000, 20)
    assertEquals(800, price)
}
```

Now, we give the task to AI: write the `calculateDiscountedPrice` function so that this test passes.

Here, the definition of done is not a matter of opinion. Either the test is green or it is red. The AI can write the function, run the test, see the failure, fix the code, and run the test again. It repeats this loop on its own until the test passes.

Here, we can notice one important thing. We did not have to trust the AI. We just had to run the check.

Many tasks fall in this category:

- **The code must compile.** The compiler gives us a clear pass or fail.
- **A function must return an exact output for an exact input.** We compare, and we know.
- **A bug must stop reproducing.** We run the steps, and the crash is either there or it is gone.

- **A SQL query must produce this exact result.** We run it and compare the rows.
- **A math answer must match the known answer.** We check it directly.

In all of these, the machine checks the machine. There is a hard boundary between done and not done, and the AI can walk right up to it by itself.

This is why AI feels magical in these cases. Give it a failing test, and it will keep working until the test passes. The checker is doing the hardest job here, telling us the truth again and again.

Tasks where the definition of done is fuzzy

Now, let's move to the other side.

We open the editor and type this: "Build the login feature."

The AI builds it. The code compiles. The screen opens. The user is able to log in.

Now, the big question. Is it done?

We do not know. And the reason is that we never said what done means.

Let's see what was hiding inside that one small sentence:

- What happens when the network fails in the middle?
- Is the token stored securely?
- What happens when the password field is empty?
- Is the error message good enough for a real user?
- Will this code be easy to change after six months?

Not a single one of these was written down. So not a single one can be checked. The AI produced something that looks complete, and looking complete is not the same as being complete.

But, here is the catch. In the unit test example, a red test told us clearly that we were not done. Here, nothing turns red. The absence of a failure is not the presence of correctness.

And there is one more trap. Sometimes the test exists and still it does not save us. If the test covers only the happy path, then green only proves that the happy path works. The AI will stop the moment the test turns green, because for the AI, green means done.

The same problem exists outside code:

- "Write a good summary for this content."
- "Design a clean architecture for this app."
- "Make this code more readable."

The words "good", "clean", and "readable" are doing all the work here, and not one of them can be checked by a machine.

So, in these tasks, the only checker left is a human being. And this is exactly why we cannot completely rely on the AI output here. Not because the AI is weak, but because nothing is standing between the output and us.

We have a detailed blog on LLM as a Judge that explains how we can use another model as the checker when no exact one exists.

Why AI is so strong exactly where we can measure

Now, the question arises. Why is this difference so sharp?

The answer is in the loop.

AI does not write the perfect answer in one shot. It works in a loop. Try something, check the result, see the mistake, fix it, try again. This loop is where the real power is.

But this loop needs one thing to keep running. After every attempt, it needs an answer to a simple question: am I done or not?

Where a checker exists, the loop runs on its own.

The AI writes the function, runs the test, sees it red, reads the error, fixes the line, and runs the test again. It can repeat this ten times or fifty times without asking us anything. Every round gives it an honest signal, and every round brings it closer. The moment the test turns green, the loop stops by itself.

So, what we finally see is not the first attempt of the AI. It is the attempt that survived many rounds of checking.

Where a checker does not exist, the loop dies after the first round.

We ask for a clean architecture. The AI writes something. Now, there is nothing to run. Nothing turns red. Nothing turns green. No signal comes back, so the AI has no reason to try again, and it stops right there.

So, what we see here is only the first attempt. One guess, and we are reading it as a final answer.

Wherever a cheap and exact checker exists, the AI gets many tries. Wherever it does not exist, the AI gets only one try.

That is the whole difference we keep feeling as developers. Same model, same intelligence, but in one case it corrected itself fifty times before showing us the output, and in the other case it showed us the output straight away.

And when the checker is missing, we become the checker. We read the output, we find the problem, we ask for a change, and the AI tries again. The loop still runs, but now it runs at our speed, and it costs our time and attention on every single round.

How to write a better definition of done

So, what do we do about it? The answer is simple to say and hard to do. **We move our tasks from the fuzzy side to the exact side.**

Let's see the practical ways.

Write the test first, then give the task. This is the biggest change we can make. The AI gets a target it can hit by itself, and we get a green or red answer instead of an opinion.

Give the exact input and expected output. Instead of "parse this date properly", we say: for the input `25-08-2026`, the output must be `2026-08-25`.

Give the command that must run clean. "The build command and the test command must both pass" is a definition of done. "Make sure it works" is not.

Turn the edge cases into actual cases. Do not write "handle the errors properly". Write the list. Empty input, network timeout, expired token, duplicate request. Every line in that list can now be checked.

Turn a vague quality into a number. "Make this API faster" cannot be checked. "The response must come under 200 ms for 1000 rows" can be checked.

We have a detailed blog on Harness Engineering in AI that explains how we build this checking layer around the model.

And then, one more step, which we usually skip.

Be honest about what is still not checkable. Some parts will always need our own judgement. Is this design right for where our product is going? No checker exists here, and pretending otherwise is how weak decisions enter our codebase quietly.

When we know which part has a checker and which part does not, we know exactly which part of the AI output we can accept and which part we must read ourselves. That is much better than reading everything with equal doubt.

The model is not the limit here. Our clarity is the limit. The sharper we can say what done means, the more work we can safely hand over.

So, before we give the next task to AI, let's ask one question first: how will I check this?

Frequently Asked Questions

What is a checker or verifier in AI coding?

A checker, also called a verifier, is anything that can look at the output and say pass or fail without any opinion. A unit test is a checker, and so is a compiler. A human reading the code is also a checker, but a

slow one that can give a different answer on a tired day.

Does a passing test mean the AI output is correct?

Not always. If the test covers only the happy path, then a green test only proves that the happy path works. The AI will stop the moment the test turns green, because for the AI, green means done. So, we turn the edge cases into actual test cases.

Is AI weak at fuzzy tasks like clean architecture?

No. The model is not weak at these tasks. The difference is that no checker exists, so no signal comes back and the AI gets only one try. What we see is just its first attempt, while on tasks with a checker it may correct itself many times first.

Can every part of a task be made checkable?

No. Some parts will always need our own judgement, like whether a design is right for where our product is going. We should be honest about what is still not checkable. Then we know which parts of the AI output we can accept and which parts we must read ourselves.

LESSON 11.5

How does LangChain work?

Lesson 11.5 · ~18 min · Published July 1, 2026 · https://outcomeschool.com/blog/how-does-langchain-work



LangChain is a framework that helps us build applications powered by Large Language Models. It sits between us and the LLM, and connects the LLM with our data, our tools, and our logic.

In this blog, we will learn about how LangChain works. We will also see why we need it, what chains, prompts, memory, and output parsers are, how retrieval and agents fit in, and how the full flow works together in the real world.

We will cover the following:

- What is LangChain?
- Why do we need LangChain?
- The core idea behind LangChain
- LLM and Prompt Template
- What is a Chain?
- Output Parser
- Memory
- Retrieval and RAG
- Tools and Agents
- A complete flow of how LangChain works

Let's get started.

What is LangChain?

LangChain is a framework that helps us build applications powered by Large Language Models.

Before we go further, let's understand one term first. A **Large Language Model**, also called an **LLM**, is a model that understands and generates human-like text. ChatGPT is built on top of an LLM. We give it some text, and it gives us back some text.

Now, LangChain is a tool that sits between us and the LLM. It helps us connect the LLM with other things like our data, our tools, and our logic.

In simple words, LangChain helps us build smart apps that talk to an LLM and do useful work for us.

Let's decompose the name to understand it better.

> LangChain = Lang + Chain

Here, `Lang` stands for Language, which points to the language model. And `Chain` means joining many steps together, one after another. So LangChain is about chaining many steps into one smooth flow.

This idea of joining prompts step by step, where the output of one prompt becomes the input of the next, is called Prompt Chaining. We have a detailed blog on How does Prompt Chaining work that explains it step by step.

Now, we have understood what LangChain is. Now, let's understand why we even need it.

Why do we need LangChain?

Let's say we want to build a simple app. We give it a question, and it gives us an answer using an LLM. This is easy. We send our text to the LLM and get the reply.

But real apps are not this simple. Let's see why.

Suppose we want our app to do the following:

- Take a question from the user.
- Look up answers from our own documents.
- Remember the previous messages in the conversation.
- Call a calculator or a search engine when needed.
- Format the final answer in a clean way.

Now, doing all of this by hand becomes messy. We have to write a lot of code to glue every piece together. We have to manage the order of steps. We have to handle the data moving from one step to the next.

This is hard to build and harder to maintain.

So, here comes LangChain to the rescue.

LangChain gives us ready-made building blocks. We just connect these blocks like Lego pieces. Each block does one job, and LangChain joins them together.

This way we can use LangChain to solve the interesting problem of building complex LLM apps in a very simple way.

Now that we have learned why we need LangChain, it's time to understand the core idea behind it.

The core idea behind LangChain

The core idea of LangChain is very simple.

> Take small building blocks and chain them together to build a complete flow.

Let's understand this with an analogy.

Consider a factory assembly line. A car is built step by step. First, the body is made. Then, the engine is added. After that, the wheels are fixed. Finally, the paint is done. Each station does one job and passes the car to the next station.

LangChain works in the same way. Each block does one small job. The output of one block becomes the input of the next block. This is the chain.

Let me map this analogy to LangChain for our better understanding.

Assembly line	LangChain
Raw material entering	User input
A single work station	A building block
The car moving forward	Data passing to the next step
The finished car	The final answer

Now, we have the big picture. Now, let's learn about each building block one by one. Do not worry, we will learn about each of them in detail.

LLM and Prompt Template

The first block is the **LLM**, which is the brain of our app. We already know that the LLM takes text and returns text.

But before sending text to the LLM, we must prepare the text carefully. The text we send to the LLM is called a **Prompt**.

In simple words, a prompt is the instruction we give to the LLM.

Now, most of the time our prompt is not fixed. A part of it changes based on the user. So, here comes the **Prompt Template** into the picture.

A Prompt Template is a ready-made prompt with some blank spaces that we fill at runtime.

Let's see the code for a prompt template as below:

```
from langchain_core.prompts import PromptTemplate
template = PromptTemplate.from_template(
    "Explain {topic} in simple words for a beginner."
)
prompt = template.format(topic="LangChain")
```

Here, we have created a template with a blank space called `topic`. When we call `format` and pass `topic="LangChain"`, the blank space gets filled. The final prompt becomes "Explain LangChain in simple words for a beginner."

This is how a prompt template helps us reuse the same prompt with different values. It makes our life easy.

Now, we have a prompt ready. Now, let's learn how to join the prompt template with the LLM. This is where the chain comes in.

What is a Chain?

A Chain is the joining of many blocks so that the output of one block flows into the next block.

In simple words, a chain is a pipeline of steps.

Let's say we want a simple chain. First, the prompt template fills in the user topic. Then, the LLM reads that prompt and gives an answer. This is a chain of two steps.

Let's see the code for a chain as below:

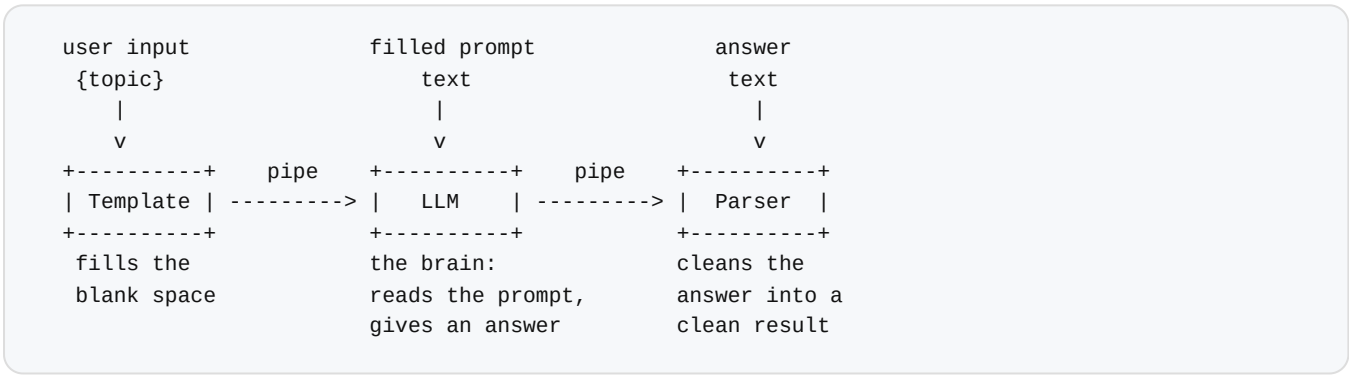
```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
llm = ChatOpenAI(model="gpt-4o-mini")
template = PromptTemplate.from_template(
    "Explain {topic} in simple words for a beginner."
)
chain = template | llm
result = chain.invoke({"topic": "LangChain"})
```

Here, we can see a few important things. We created the `llm` block and the `template` block. Then we joined them using the `|` symbol. This `|` symbol is the pipe, and it means "send the output of the left side into the right side."

So, `template | llm` means the filled prompt from the template goes straight into the LLM. When we call `invoke` with our topic, the data flows through the chain and we get a result.

This is the heart of LangChain. We build small blocks and join them with the pipe.

Let's visualize this chain as below:



Here, we can see that the data flows from left to right. The top row shows the data on each wire, and the bottom row shows what each block does. The `Template` fills the blank space and passes the filled prompt to the `LLM`. The `LLM` is the brain. It reads the prompt and passes its answer forward to the `Parser`. The `|` pipe between each block is what carries the output of one block into the next block. This is exactly what a chain means.

Now, we have understood the chain. But the LLM gives us plain text. Many times we need the answer in a clean structure. So, let's learn about the output parser.

Output Parser

An Output Parser takes the raw text from the LLM and converts it into a clean, structured form.

In simple words, the LLM speaks in plain text, and the output parser turns that text into something our program can use easily.

Let's say we ask the LLM for a list of three fruits. The LLM returns "apple, banana, mango" as plain text. But in our code, we need an actual list of three separate items. The output parser does this

conversion for us.

Let's see the code for an output parser as below:

```
from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()
chain = template | llm | parser
result = chain.invoke({"topic": "LangChain"})
```

Here, we have added one more block called `parser` to our chain. Now the data flows through three blocks. The template fills the prompt, the LLM gives an answer, and the parser cleans that answer into a simple string. The `StrOutputParser` simply gives us back clean text instead of the full message object.

This is how an output parser makes the final result neat and ready to use.

Now, our chain can take input, talk to the LLM, and clean the output. But there is a problem. Our app forgets everything after every message. Let's solve this with memory.

Memory

Let's say we are chatting with our app. We ask, "Who is Albert Einstein?" The app answers correctly. Then we ask, "Where was he born?"

But here is the catch. The app does not know who "he" is, because it forgot the previous message. Every call to the LLM is fresh, and it does not remember the past on its own.

So, here comes **Memory** to the rescue.

Memory is the part of LangChain that stores the previous messages of a conversation so the app can remember the context.

In simple words, memory gives our app the power to recall what was said before.

Let's understand how it works. Each time the user sends a new message, LangChain takes the old messages from memory and adds them to the new prompt. So the LLM sees the full conversation, not just the latest message.

Let's see a simple flow of memory as below:

```
# Turn 1
# Stored in memory: "User: Who is Albert Einstein?"
# Turn 2
# Memory + new message are sent together:
# "User: Who is Albert Einstein?
#  AI: Albert Einstein was a famous physicist.
#  User: Where was he born?"
```

Here, we can notice that in the second turn, the old conversation is added before the new question. So now the LLM knows that "he" means Albert Einstein. The memory carries the context forward.

This is how memory makes our app feel like a real conversation. Problem Solved.

Now, our app can remember the chat. But what if we want the app to answer using our own documents? The LLM does not know about our private data. Let's solve this with retrieval.

Retrieval and RAG

The LLM is trained on general data from the internet. It does not know about our company documents, our personal notes, or our latest files.

Let's say we have a 200-page company handbook. We want our app to answer questions from this handbook. The LLM has never seen it. So how do we teach the app about it?

So, here comes **Retrieval** into the picture.

Retrieval means fetching the most relevant pieces of our own data and giving them to the LLM along with the question.

This full pattern has a name. It is called **RAG**, which stands for **Retrieval-Augmented Generation**. In simple words, we first retrieve the right information, then we let the LLM generate the answer using that information.

Let's understand the steps of how retrieval works.

Step 1: We break our big document into small pieces called chunks.

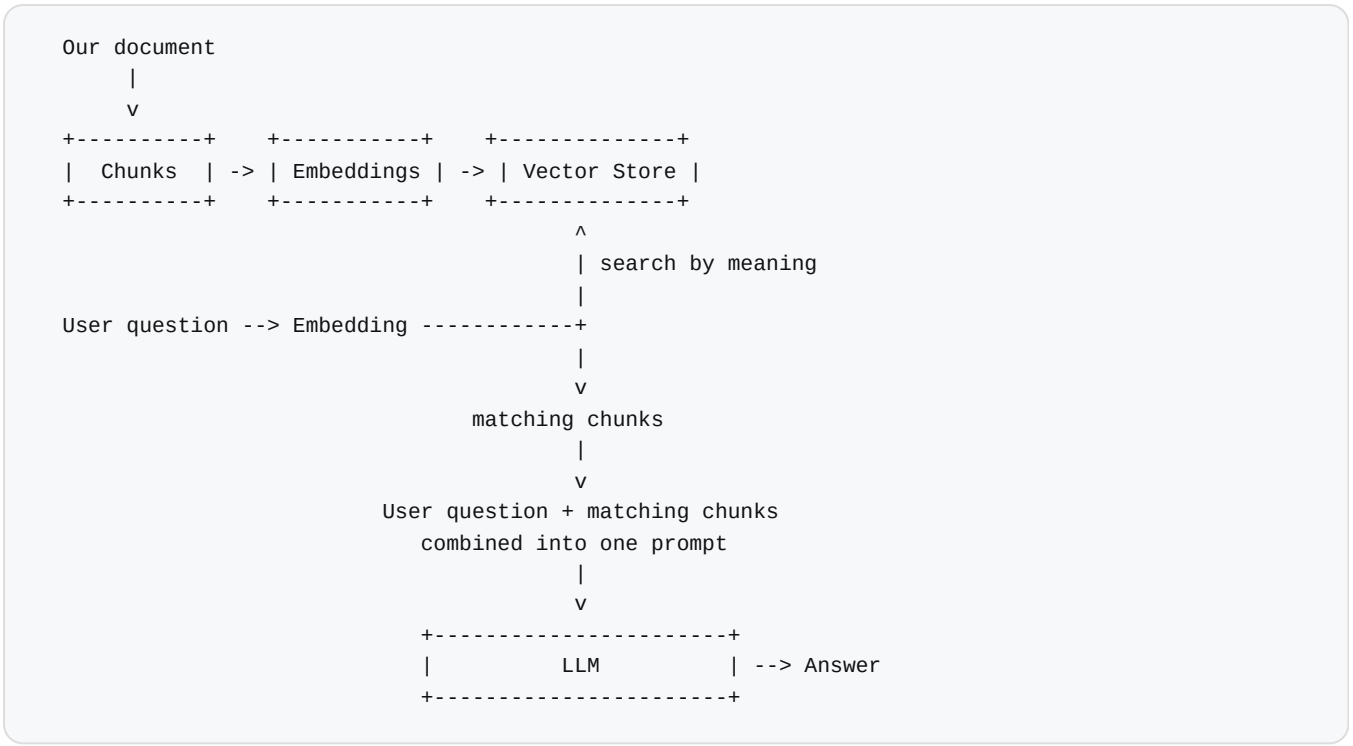
Step 2: We convert each chunk into a list of numbers called an **embedding**. An embedding is just a way to represent text as numbers so the computer can compare meaning.

Step 3: We store all these embeddings in a special database called a **Vector Store**. A vector store is a database built to search by meaning, not just by exact words.

Step 4: When the user asks a question, we turn the question into an embedding too, and we search the vector store for the closest matching chunks.

Step 5: We send those matching chunks plus the question to the LLM, and the LLM answers using them.

We can picture the RAG flow as below:



Here, we can see that the top row prepares our data once. We split the document into chunks, turn each chunk into an embedding, and store all of them in the `Vector Store`. When the user asks a question, we turn the question into an embedding too and search the `Vector Store` by meaning. The matching chunks and the question are then combined into one single prompt, and that single prompt is sent to the `LLM`, which generates the final answer from our own data.

Let's see a simplified code for retrieval as below:

```
# Find the chunks that match the user question
relevant_chunks = vector_store.similarity_search("What is the leave policy?")
# Send the chunks and the question to the LLM
chain = prompt_with_context | llm | parser
result = chain.invoke({
    "context": relevant_chunks,
    "question": "What is the leave policy?"
})
```

Here, we can see that we first search the vector store for chunks related to the leave policy. Then we pass both the chunks and the question into our chain. The LLM now reads our own data and answers from it.

This is how retrieval and RAG let the LLM answer questions from our private data. That's the beauty of LangChain.

We have a detailed blog on Agentic RAG that takes this retrieval pattern further.

Now, our app can use memory and our own documents. But sometimes the LLM needs to take action in the real world, like searching the web or doing math. Let's learn about tools and agents.

Tools and Agents

The LLM is very good at language, but it is not good at everything. For example, the LLM is weak at exact math, and it does not know today's live information.

Let's say we ask, "What is the weather in Delhi right now?" The LLM cannot know this, because it has no live connection to the weather.

So, here comes the idea of **Tools** to the rescue.

A Tool is an external function, such as a calculator, a search engine, or a weather service, that can be run to do something the LLM cannot do on its own.

In simple words, a tool is a helper that gets run when the LLM cannot do the job by itself.

Now, here is one very important point that people often get wrong. The LLM does not run the tool by itself. The LLM only recommends which tool to use and what input to pass to it. The actual call to the tool is made by our code. The LLM is the advisor, and our code is the doer.

So, the next big question is: who takes the LLM's recommendation and actually runs the tool? This is where the **Agent** comes in.

An Agent is a smart controller that uses the LLM to decide which tool to use and in what order, then runs that tool, takes back the result, and gives it to the LLM, until the task is complete.

In simple words, the agent works in a loop. It asks the LLM what to do next. The LLM recommends a tool and the input for it. The agent runs that tool, sees the result, and sends the result back to the LLM. Then it asks the LLM for the next step again. It keeps doing this until the task is done.

Let's see a numeric walkthrough of how an agent works. Suppose the user asks, "What is 25 times 18, and is that more than 400?"

Step 1: The agent sends the question to the LLM. The LLM recommends the calculator tool with the input `25 * 18`.

Step 2: The agent runs the calculator tool with `25 * 18`. The tool returns `450`.

Step 3: The agent sends `450` back to the LLM. The LLM compares `450` with `400` and sees that `450` is more than `400`.

Step 4: The LLM forms the final answer: "25 times 18 is 450, which is more than 400."

Here, we can notice that the math was not guessed. The agent ran a real tool for the calculation, and then the LLM reasoned about the result. This makes the answer reliable.

When these agent steps, decisions, and loops grow complex, we can lay them out as a graph and let the framework run them for us. We have a detailed blog on how LangGraph works that explains this step by step.

Note: The agent uses the LLM as its brain to make decisions, and the tools as its hands to take actions.

This is how tools and agents give our app the power to do real work.

Now, we have learned about every building block. Now, let's put them all together into one complete flow.

A complete flow of how LangChain works

Let's connect everything we have learned into a single story. Suppose a user opens our app and asks a question about our company handbook.

Step 1: The user types the question into our app.

Step 2: LangChain pulls the previous messages from memory and adds them for context.

Step 3: LangChain uses retrieval to fetch the matching chunks from the vector store.

Step 4: The prompt template combines the question, the memory, and the retrieved chunks into one final prompt.

Step 5: This prompt flows into the LLM through the chain.

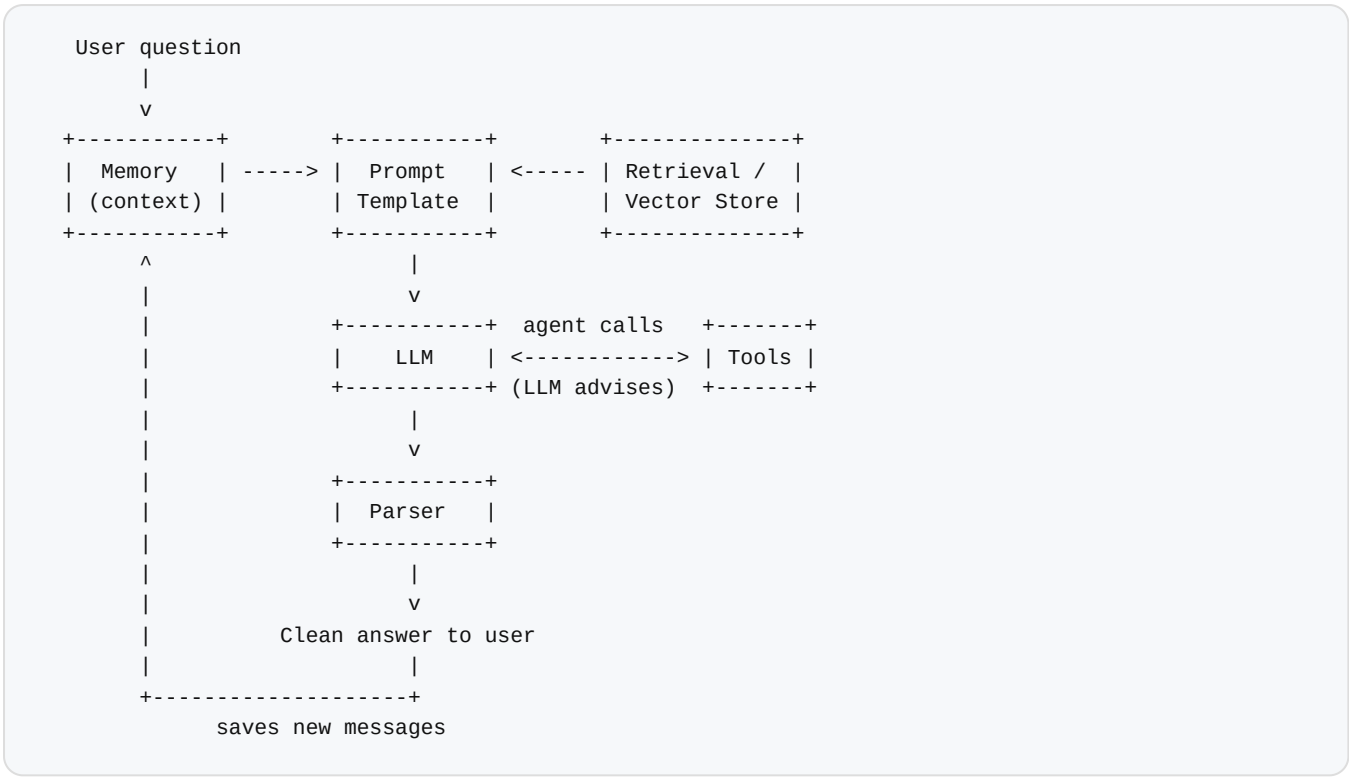
Step 6: If the task needs an action, the LLM recommends a tool, and the agent runs that tool and brings the result back to the LLM.

Step 7: The LLM produces an answer.

Step 8: The output parser cleans the answer into a neat form.

Step 9: The clean answer is shown to the user, and the new messages are saved into memory for the next turn.

The complete flow looks like below:



Here, we can see how all the blocks work together. The question first gathers context from `Memory` and matching chunks from `Retrieval`. The `Prompt Template` combines everything into one prompt and sends it to the `LLM`. When an action is needed, the `LLM` recommends a tool, and the `Agent` runs that tool and feeds the result back to the `LLM`. The answer then passes through the `Parser` to become clean, and finally goes back to the user while the new messages are saved into `Memory` for the next turn.

Here, we can see how each block does one job and passes its result to the next block. The input flows in from one side, moves through the chain, and the final answer comes out from the other side. This is exactly the assembly line we talked about at the start.

This is how LangChain works, end to end. Let's quickly recap what we have learned.

LangChain is a framework that helps us build applications powered by LLMs. It gives us small building blocks and lets us chain them together.

We learned about the prompt template that prepares the text, the chain that joins the blocks, and the output parser that cleans the result. We learned about memory that remembers the conversation, retrieval and RAG that bring in our own data, and tools and agents that take real actions.

Now, we must have understood how LangChain works.

Frequently Asked Questions

Is LangChain an LLM?

No. LangChain is not an LLM. It is a framework that sits between us and the LLM. It gives us ready-made building blocks, like prompt templates, chains, output parsers, memory, retrieval, and tools, and helps us connect the LLM with our data, our tools, and our logic.

Does the LLM run tools by itself in LangChain?

No. The LLM only recommends which tool to use and what input to pass to it. The actual call to the tool is made by our code through the agent. The agent runs the tool, takes back the result, and gives it to the LLM, until the task is complete.

Does an LLM remember earlier messages without memory?

No. Every call to the LLM is fresh, and it does not remember the past on its own. LangChain memory stores the previous messages and adds them to each new prompt, so the LLM sees the full conversation and knows, for example, who "he" refers to.

What does the pipe symbol do in a LangChain chain?

The `|` pipe sends the output of the left side into the right side. For example, `template | llm | parser` means the filled prompt goes into the LLM, and the LLM's answer goes into the parser. This is how we join small blocks into one chain.

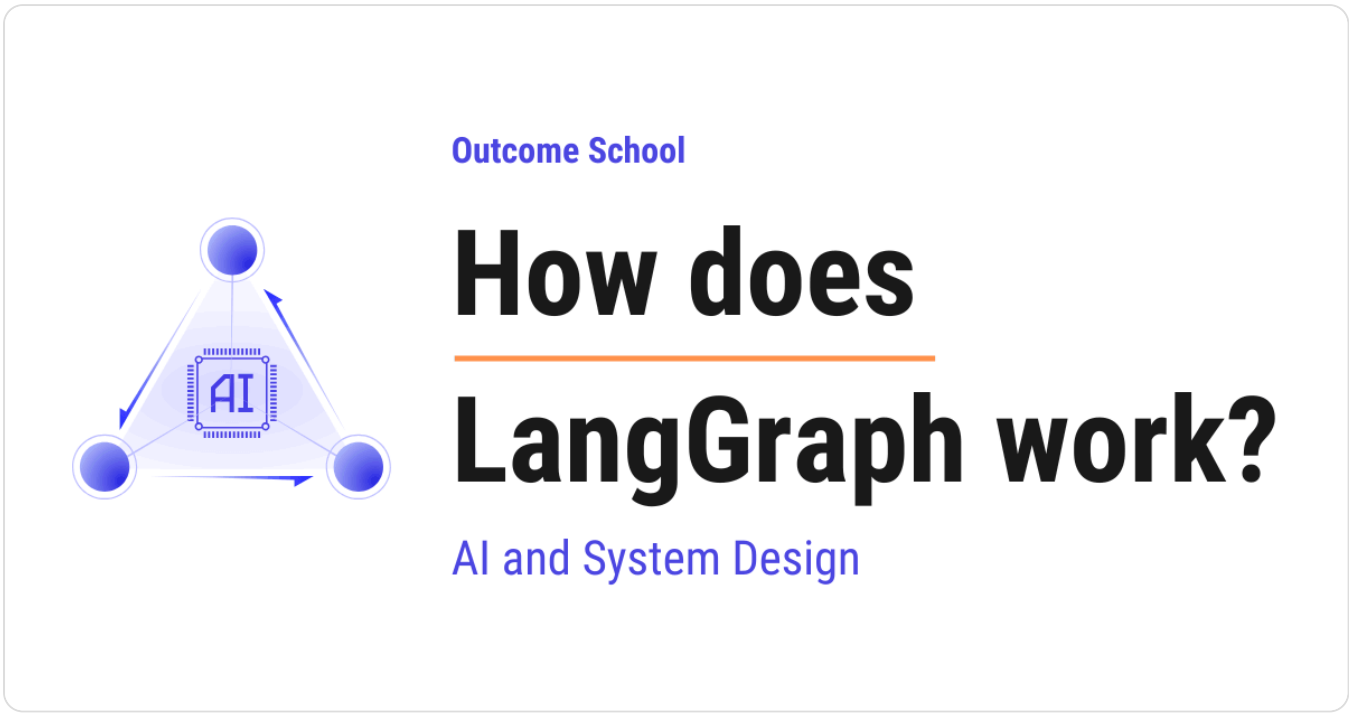
When should we use LangGraph instead of a simple LangChain chain?

We should use LangGraph when agent steps, decisions, and loops grow complex. In that case, we can lay them out as a graph and let the framework run them for us.

LESSON 11.6

How does LangGraph work?

Lesson 11.6 · ~19 min · Published July 2, 2026 · https://outcomeschool.com/blog/how-does-langgraph-work



LangGraph is a framework that helps us build applications powered by an LLM, where the work is organized as a graph of steps, and a shared state travels from one step to the next.

In this blog, we will learn about how LangGraph works. We will also see why we need it, what graphs, state, nodes, and edges are, how tools work and who actually calls them, how memory and human-in-the-loop fit in through a complete example, and when to use it in the real world.

We will cover the following:

- What is LangGraph?
- Why do we need LangGraph?
- What is a Graph in LangGraph?
- What is State in LangGraph?
- Nodes and Edges
- Conditional Edges
- A complete example
- Tools and who calls them
- Memory and persistence
- Human-in-the-loop
- When to use LangGraph

Let's get started.

What is LangGraph?

LangGraph is a framework that helps us build applications powered by an LLM, where the work is organized as a graph of steps.

Before we go ahead, let's understand one term first. An **LLM** is a Large Language Model. It is the kind of AI model that powers tools like ChatGPT. We give it some text, and it gives us back some text.

Now, let's decompose the name to make it simple.

LangGraph = Lang + Graph

`Lang` comes from language, because we are working with language models. `Graph` is a way of connecting steps together. So, LangGraph is a way to connect language model steps together in a structured manner.

In simple words, LangGraph lets us describe an AI application as a set of steps, and it controls which step runs, in what order, and what happens next.

LangGraph comes from the same team behind LangChain and builds on its ideas, adding graphs, loops, and branching on top. We have a detailed blog on how LangChain works that explains this in depth.

Why do we need LangGraph?

The best way to learn this is by taking an example.

Suppose we want to build a simple AI app. We send a question to the LLM, and it gives us an answer. This is easy. We do not need any framework for this.

But real applications are not that simple. Let's say we are building a customer support assistant. The assistant must do many things.

First, it must read the user's message. Then, it must decide whether the question is about billing or about a technical issue. After that, it must search the right document. Then, it must ask the LLM to write a reply. And sometimes, it must ask a human to approve the reply before sending it.

Now, here is the catch. These steps are not a straight line. The app must make decisions. It must loop back and try again. It must remember what happened before. It must pause and wait for a human.

If we write all of this with plain `if` and `else` conditions, the code becomes messy very fast. It becomes hard to read and hard to change.

So, here comes LangGraph to the rescue.

LangGraph lets us describe these steps and decisions clearly as a graph. It then runs the graph for us. It makes our life easy.

What is a Graph in LangGraph?

Before jumping into LangGraph, we must know what a graph is.

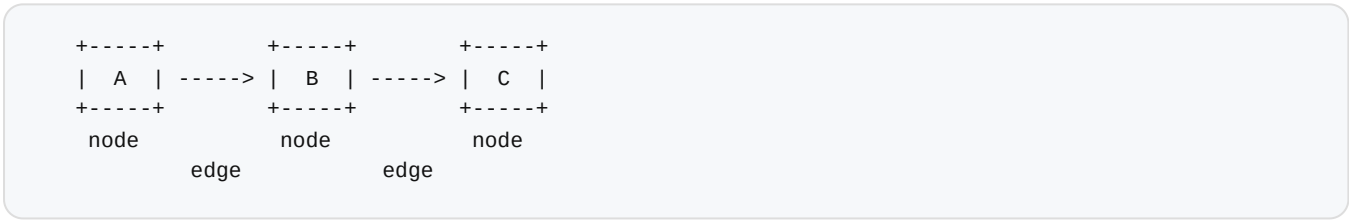
A graph is a collection of points connected by lines.

In simple words, a graph has two things.

- **Nodes**: these are the points. Each point is a piece of work.
- **Edges**: these are the lines. Each line connects one point to the next point.

Let's say we draw a small map. We have three circles named A, B, and C. We draw an arrow from A to B, and another arrow from B to C. This is a graph. The circles are nodes. The arrows are edges.

Let's visualize this as below:



Here, we can see that `A`, `B`, and `C` are the nodes, which are the boxes that do work. The arrows between them are the edges, which tell the graph where to go next. The graph starts at `A`, moves to `B`, and then moves to `C`.

Consider a simple real-world map. We start at home, then we go to the office, then we go back home. Home and office are nodes. The roads between them are edges.

In LangGraph, each node does some work, and each edge tells the app where to go next. This is the core idea, and everything else is built on top of it.

Designing a system this way, as nodes and edges, is called Graph Engineering, and we have a detailed blog on it that explains this from the ground up.

What is State in LangGraph?

Before we build a graph, we must understand one more important concept called state.

State is the shared memory that travels through the graph.

In simple words, the state is a box of information that every node can read from and write to. As the app moves from one node to the next, the state carries the data along.

Let's say we are processing an order. The state can hold the user's message, the items in the cart, and the final reply. Node 1 puts the user's message into the state. Node 2 reads that message and puts the cart items into the state. Node 3 reads everything and writes the final reply.

This way, the nodes do not need to call each other directly. They just read and write to the shared state. The state is the glue that holds the whole graph together.

Let's see the code for a simple state as below:

```
from typing import TypedDict
class State(TypedDict):
    message: str
    reply: str
```

Here, we have defined a `State` using `TypedDict`. A `TypedDict` is just a dictionary where we declare the names and the types of the values it will hold. Our state holds two things. The `message` is the user's input. The `reply` is the answer we will produce. This is the box that will travel through our graph.

Nodes and Edges

Now that we have understood the state, it's time to learn about nodes and edges in LangGraph.

A node is a Python function that takes the current state and returns an update to the state.

In simple words, a node is one step of work. It reads the state, does something, and gives back the part of the state it wants to change.

Let's see the code for a node as below:

```
def write_reply(state: State):
    user_message = state["message"]
    answer = "Thanks for your message: " + user_message
    return {"reply": answer}
```

Here, we have created a node named `write_reply`. It reads the `message` from the state. It builds a small answer. Then it returns a dictionary with the new `reply`. LangGraph takes this returned value and updates the shared state with it. The node does not change anything else. It only updates what it returns.

Now, let's understand edges.

An edge connects one node to the next node, and tells the graph where to go after a node finishes.

In simple words, an edge is the arrow between the steps. After one node finishes, the edge decides which node runs next.

There are two special points in every graph. The `START` point is where the graph begins. The `END` point is where the graph stops. We connect our first node to `START` and our last node to `END`.

Conditional Edges

Till now, we have learned about normal edges that always go to the same next node. But real apps must make decisions.

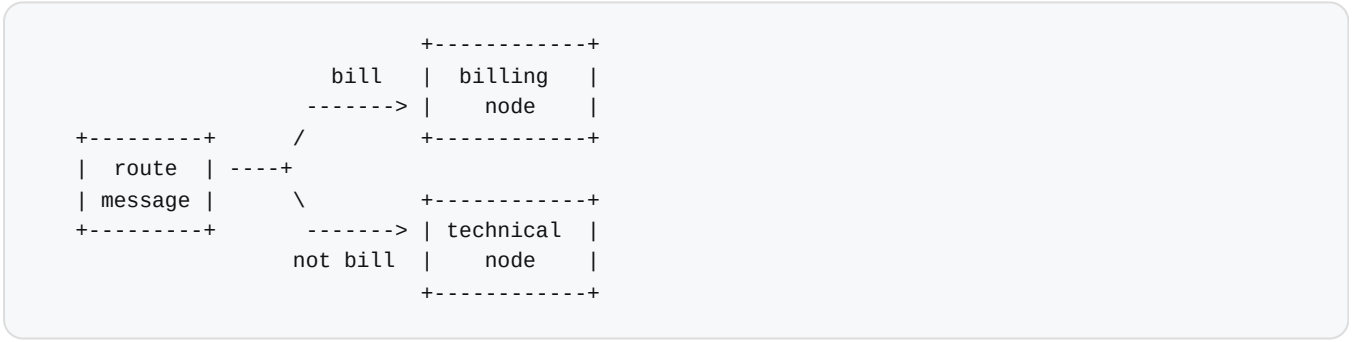
So, here comes the **conditional edge** into the picture.

A conditional edge is an edge that picks the next node based on the current state.

In simple words, a conditional edge is like a fork on a road. Based on what is in the state, the graph chooses one path or another.

Let's say a user sends a message. We must check whether it is a billing question or a technical question. If it is billing, we go to the billing node. If it is technical, we go to the technical node. This decision is made by a conditional edge.

We can picture the fork as below:



Here, we can see that the `route message` step looks at the state and then picks one path. If the message contains the word `bill`, the graph goes to the `billing` node. Otherwise, the graph goes to the `technical` node. This is how a conditional edge branches the graph based on the state.

Let's see the code for a conditional edge function as below:

```
def route_message(state: State):
    if "bill" in state["message"]:
        return "billing"
    return "technical"
```

Here, we have created a function named `route_message`. It reads the `message` from the state. If the message contains the word `bill`, it returns the text `billing`. Otherwise, it returns the text `technical`. LangGraph reads this returned text and sends the graph to the matching node. This way, the graph can branch in different directions.

A complete example

Now, the best way to bring everything together is by building the same billing example from start to end.

Let's recall our customer support assistant. It reads a user message, decides whether the message is about billing or about a technical issue, and then replies with the right team. We have already seen the state and the routing function. Now, let's assemble the full graph step by step.

First, we define the state and the two nodes as below:

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
class State(TypedDict):
    message: str
    reply: str
def billing(state: State):
    return {"reply": "This is the billing team. We will help you with your payment."}
def technical(state: State):
    return {"reply": "This is the technical team. We will help you fix the issue."}
```

Here, we have defined our `State` with `message` and `reply`. We have created two nodes. The `billing` node writes a reply from the billing team. The `technical` node writes a reply from the technical team. We have also imported `StateGraph`, which is the builder we use to create our graph, along with `START` and `END`, which are the start and stop points.

Then, we bring back our routing function as below:

```
def route_message(state: State):
    if "bill" in state["message"]:
        return "billing"
    return "technical"
```

Here, we have the same `route_message` function we saw earlier. It reads the `message` from the state. If the message contains the word `bill`, it returns `billing`. Otherwise, it returns `technical`. This is the function that decides which node runs next.

After that, we build the graph and connect the parts as below:

```
builder = StateGraph(State)
builder.add_node("billing", billing)
builder.add_node("technical", technical)
builder.add_conditional_edges(START, route_message)
builder.add_edge("billing", END)
builder.add_edge("technical", END)
graph = builder.compile()
```

Here, we can see the full wiring. We create a `StateGraph` and give it our `State`. We add our two nodes with `add_node`. Then we add a conditional edge from `START` using `add_conditional_edges`,

which calls `route_message` to pick the next node. We add an edge from `billing` to `END` and from `technical` to `END`, so the graph stops after it replies. Finally, we call `compile`, which turns our description into a runnable graph.

After that, we run the graph with a billing message as below:

```
result = graph.invoke({"message": "I have a question about my bill"})
print(result["reply"])
```

Here, we have called `invoke` and passed the starting state with a `message` about a bill. The graph reads the message, the conditional edge sends it to the `billing` node, the node writes the reply, and we get the final state. We print the `reply` from it.

This will print the following:

```
This is the billing team. We will help you with your payment.
```

It works perfectly. The message had the word `bill`, so the graph picked the billing path on its own.

Now, let's run the graph with a different message as below:

```
result = graph.invoke({"message": "My app is not working"})
print(result["reply"])
```

This will print the following:

```
This is the technical team. We will help you fix the issue.
```

This time the message did not have the word `bill`, so the conditional edge sent the graph to the `technical` node. This is the same billing example we have been building, now running from start to end.

We can picture the full flow as below:

```
                          +------------+
                  bill    |  billing   | ---> END
                -------> |    node    |
+-------+       /         +------------+
| START | ----->+
+-------+       \         +------------+
                 -------> | technical  | ---> END
                not bill  |    node    |
                          +------------+
```

Here, we can see that the graph begins at `START`. The conditional edge reads the `message` from the state and picks one path. If the message contains `bill`, the graph goes to the `billing` node.

Otherwise, it goes to the `technical` node. The chosen node writes the `reply` into the state, and the graph moves to `END`. We get the final state back with both the `message` and the `reply` filled in.

Tools and who calls them

Now, it's time to learn about a very important concept called tools. This is also the part where we often get confused, so we must read it carefully.

First, let's remember what an LLM really is. **An LLM only takes text and gives back text.** It cannot search the web on its own. It cannot read our database. It cannot send an email. It can only produce words.

But real apps need real actions. We may need to search the web, look up an order, or check the weather. So, here comes the **tool** into the picture.

A tool is a normal function that does a real action, like searching the web or reading a database.

Now, here is the catch, and we must be very clear about it.

The LLM does not call the tool. The LLM only recommends which tool to use.

In simple words, the LLM reads the user's request and says, "We should use the search tool, and the input should be weather in Delhi." That is all the LLM does. It returns this recommendation as text. It does not run anything.

We can picture the LLM's recommendation as below:

```
tool:  search
input: weather in Delhi
```

Here, we can see that the LLM only produced a small piece of text. It named the tool and the input. It did not run the search. It just gave a recommendation.

So, now the question is, who actually calls the tool? The answer is the **agent**.

In simple words, the agent is the code that runs around the LLM. Here, LangGraph is that agent. It reads the recommendation from the LLM, calls the real tool function, gets the result, and then gives the result back to the LLM.

Let's understand this with a simple analogy. Think of the LLM as a doctor and the agent as a pharmacist. The doctor looks at the patient and writes a prescription. The doctor only recommends the medicine. The doctor does not hand over the medicine. The pharmacist reads the prescription and actually gives the medicine. In the same way, the LLM only recommends the tool, and the agent actually calls it.

Let's see this flow step by step.

- **Step 1:** The user asks a question.

- **Step 2:** The LLM reads the question and recommends a tool with some input. It does not run the tool.
- **Step 3:** The agent reads this recommendation and actually calls the tool.
- **Step 4:** The tool runs and returns a result.
- **Step 5:** The agent gives the result back to the LLM.
- **Step 6:** The LLM reads the result and writes the final answer.

We can picture this flow as below:

```
user question
     |
     v
+-----------+
| LLM node  |   recommends a tool (but does not call it)
+-----------+
     |
     v
+-----------+
|   agent   |   reads the recommendation
+-----------+
     |
     v
+-----------+
| tool node |   actually calls the tool, gets the result
+-----------+
     |
     v   the result goes back to the LLM node
+-----------+
| LLM node  |   reads the result and writes the final answer
+-----------+
```

Here, we can see the clear split of work. The LLM node only thinks and recommends. The tool node only does the real action. The agent connects the two. The LLM recommends, and the agent calls. This is the key idea, and we must always keep it in mind.

In LangGraph, this fits perfectly with what we have already learned. The LLM is one node. The tool is another node. After the LLM node, a conditional edge checks one simple thing: did the LLM recommend a tool? If yes, the graph goes to the tool node, runs the tool, and loops back to the LLM node with the result. If no, the graph goes to `END` with the final answer.

This is why LangGraph is so useful for building agents. It gives us a clean way to run this loop of recommend, call, and respond.

This pattern, where the LLM names a tool and its input and the agent runs it, is called function calling. We have a detailed blog on how function calling works in LLMs that covers this end to end.

Memory and persistence

Now, let's understand a very useful feature called memory.

Memory in LangGraph means the graph can remember the state across many runs.

In simple words, without memory, the graph forgets everything once it finishes. With memory, the graph can save the state and continue later from the same place.

Let's say we are building a chat assistant. The user says "My name is Amit". Later, the user asks "What is my name?". For the second question to work, the assistant must remember the first message. This is where memory comes in.

In LangGraph, we add memory using something called a **checkpointer**. A checkpointer is a small helper that saves the state after each step. When the user comes back, the graph loads the saved state and continues.

This way, our app can hold long conversations and pick up exactly where it left off. The state is no longer lost. It is persisted, which means it is stored safely.

Human-in-the-loop

Now, it's time to learn about another powerful feature called human-in-the-loop.

Human-in-the-loop means the graph can pause and wait for a human to check or approve a step before moving ahead.

In simple words, sometimes we do not want the AI to act fully on its own. We want a person to review the work first.

Let's say our assistant wrote a reply to a customer, but the reply involves a refund. We do not want to send a refund automatically. So, the graph pauses, a human reviews the reply, and only after approval does the graph continue.

Because LangGraph saves the state with a checkpointer, it can stop at any node and wait. The human checks the state, makes a decision, and the graph resumes from that exact point. This gives us control and safety in important workflows.

When to use LangGraph

Now, we have understood how LangGraph works. Let's understand when we must use it.

Let me tabulate the differences between a simple LLM call and LangGraph for your better understanding so that you can decide which one to use based on your use case.

Situation	Simple LLM call	LangGraph
One question, one answer	Best fit	Not needed
Many steps in order	Hard to manage	Easy to manage
Decisions and branching	Messy with `if` and `else`	Clean with conditional edges
Looping back to retry	Difficult	Built-in support
Calling tools or actions	LLM can only recommend	Agent calls the tool
Remembering past runs	Manual work	Checkpointer handles it
Pausing for a human	Very hard	Human-in-the-loop support

Here, we can see the pattern clearly. For a single question and a single answer, a plain LLM call is enough. But when our app has many steps, decisions, loops, tools, memory, and human checks, LangGraph makes our life easier.

So, now we know where we can use LangGraph.

Let's quickly recall the full flow with a simple analogy table. This maps each LangGraph piece to a real-world role so that it stays in our memory.

LangGraph piece	Real-world role
State	The shared notebook everyone writes in
Node	A worker doing one task
Edge	The path from one worker to the next
Conditional edge	A fork on the road
Tool	A real action like searching the web
LLM	The advisor who only recommends the tool
Agent	The worker who actually calls the tool
Checkpointer	The save button
Human-in-the-loop	The manager who approves the work

Here, we can notice how each technical piece maps to something we already know from daily life. The state is the shared notebook. The nodes are workers. The edges are the paths. The conditional edge is a fork. The tool is a real action. The LLM is the advisor who only recommends the tool, and the agent is the worker who actually calls it. The checkpointer is the save button. The human-in-the-loop is the manager who approves.

This is how LangGraph works. We describe our AI application as a graph of nodes and edges, we let a shared state carry the data, we use conditional edges to make decisions, and we use a checkpointer for memory and human approval. We must also remember the key point about tools. The LLM only recommends which tool to use, and the agent actually calls the tool. The framework runs the graph for us, and we get a clear and powerful way to build complex AI applications in a very simple manner.

Frequently Asked Questions

What is the difference between LangGraph and LangChain?

LangGraph comes from the same team behind LangChain and builds on its ideas. It adds graphs, loops, and branching on top. In LangGraph, we describe the app as nodes and edges, a shared state carries the data, and the framework decides which step runs next.

Do we need LangGraph for a simple question and answer app?

No. For one question and one answer, a plain LLM call is the best fit, and we do not need any framework. LangGraph becomes useful when our app has many steps, decisions, loops, tool calls, memory, or human approval.

Do nodes in LangGraph call each other directly?

No. Each node reads from the shared state, does its work, and returns only the part of the state it wants to change. The edges then decide which node runs next. The state is the glue that holds the whole graph together.

Can a LangGraph graph loop back to an earlier step?

Yes, looping back is built in. For example, after the LLM node, a conditional edge checks whether the LLM recommended a tool. If yes, the graph goes to the tool node, runs the tool, and loops back to the LLM node with the result. If no, it goes to `END`.

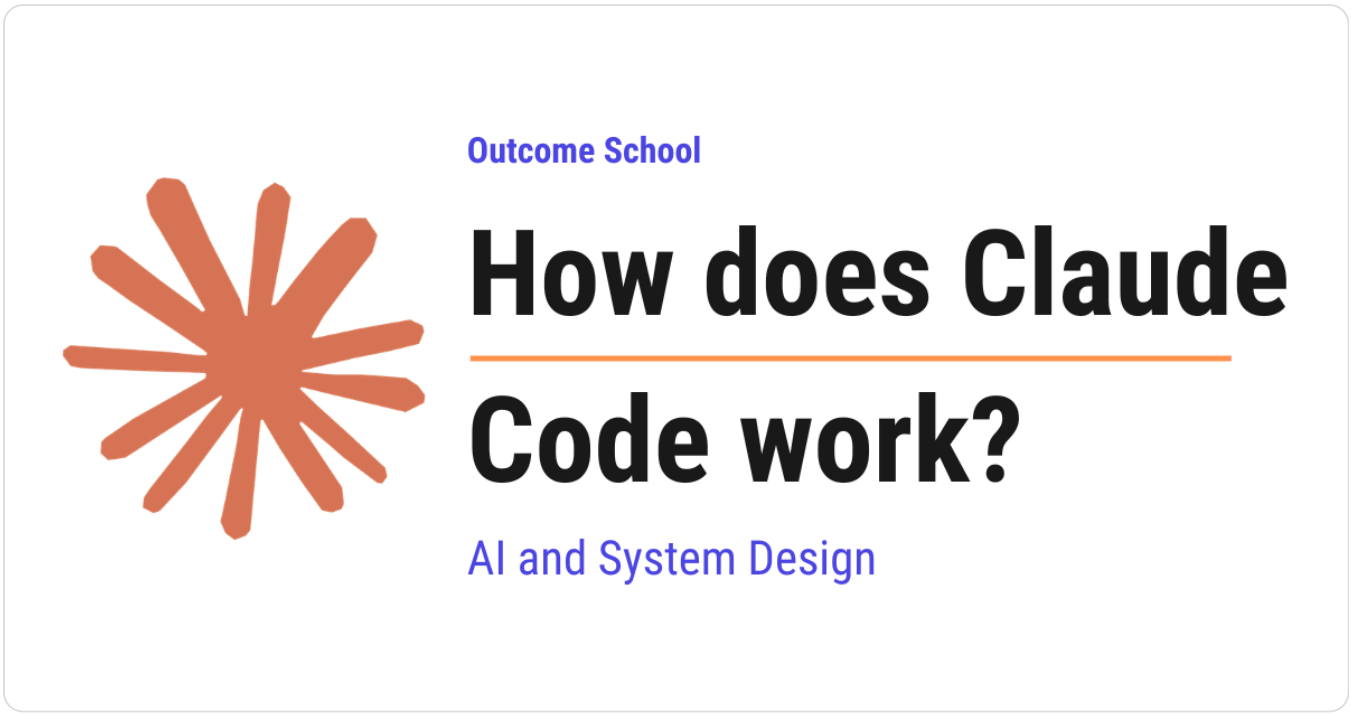
Why does human-in-the-loop depend on the checkpointer?

Because the checkpointer saves the state after each step. This lets the graph stop at any node and wait. A human checks the saved state, makes a decision, and the graph resumes from that exact point.

LESSON 11.7

How does Claude Code work?

Lesson 11.7 · ~18 min · Published July 20, 2026 · https://outcomeschool.com/blog/how-does-claude-code-work



Claude Code is a coding agent from Anthropic that runs in the terminal. We give it a task in plain English, and it completes the task by reading our code, editing files, running commands, and checking its own work.

In this blog, we will learn about how Claude Code works. We will also see what Claude Code is, why a normal AI chatbot is not enough, how the agent loop drives it, what tools give it its eyes and hands, how it searches a big project and verifies its own work, and how CLAUDE.md, permissions, plan mode, subagents, and hooks fit together.

We will cover the following:

- What is Claude Code?
- The problem with a normal AI chatbot
- The agent loop
- The tools of Claude Code
- Example: Claude Code fixing a bug
- How does Claude Code search a big project?
- How does Claude Code verify its own work?
- CLAUDE.md: The project memory
- Permissions: How we stay in control
- Plan mode, subagents, and hooks

- Putting it all together

Let's get started.

What is Claude Code?

Claude Code is a coding agent from Anthropic that runs in the terminal. We give it a task in plain English, and it completes the task by reading our code, editing files, running commands, and checking its own work.

Before we go further, we must understand two words used above: terminal and agent.

The **terminal** is the text window on our computer where we type commands. There are no buttons and no icons. We type a command, press enter, and the computer does the work and shows the result as text.

An **agent** is a program that does not just answer our question, it takes actions on its own to complete a goal.

In simple words, Claude Code is like a developer sitting inside our computer. We tell it what we want in normal English, and it does the actual work: it opens files, reads code, makes changes, runs the code, and confirms that everything works.

Let's say we hire a new developer for our team. On day one, we tell the developer: "There is a bug in the login page, please fix it." The developer does not ask us to copy and paste every file for them. They open the project, search for the login code, read it, fix it, run the tests, and tell us it is done. Claude Code works exactly like this developer.

At the center of Claude Code is **Claude**, an AI model created by Anthropic, trained on a huge amount of text and code, so it can read and write both. But a model alone can only talk. To understand why that is a problem, let's look at how we use a normal AI chatbot.

The problem with a normal AI chatbot

Suppose we have a bug in our project and we ask a normal AI chatbot for help.

First, we copy the code from our project and paste it into the chat. Then, the chatbot replies with the fixed code. After that, we copy the fixed code back into our project. Finally, we run the tests ourselves to see if the fix works.

If the fix does not work, we copy the error, paste it into the chat, and repeat the whole cycle again.

Here, the chatbot has three big limitations:

- It cannot see our project. A real project has hundreds or thousands of files, and the chatbot only sees the small piece we paste.

- It cannot do anything. It can only produce text. We have to make every change with our own hands.
- It cannot verify its answer. It never knows if the code it suggested actually works.

We are working as a messenger between the chatbot and our computer. This is slow and painful.

The whole situation looks like below:

```
+--------------+   copy and paste   +--------+   copy and paste   +--------------+
|  AI chatbot  | <----------------> |   We   | <----------------> | Our computer |
|  only talks  |                    |        |                    |  runs code   |
+--------------+                    +--------+                    +--------------+
```

Here, we can see that we are sitting in the middle. The chatbot cannot reach our computer, and our computer cannot reach the chatbot. Everything passes through our hands, one copy and paste at a time.

So, here comes Claude Code to the rescue.

Claude Code gives the AI model the two things it never had: eyes and hands. The eyes and hands are called **tools**. The model is the brain that decides what to do, and the tools are what actually do it. Do not worry, we will learn about each tool in detail.

The agent loop

The heart of Claude Code is the **agent loop**.

When we give Claude Code a task, it works through three phases: **gather context**, **take action**, and **verify results**. And, it repeats these phases again and again until the task is complete.

The agent loop looks like below:



Here, we can see the three phases of the loop:

- **Gather context:** Claude Code searches and reads the files of our project to understand the code before touching it.
- **Take action:** It edits files, creates new files, and runs commands to make the change.
- **Verify results:** It runs the tests or the program to check if the change actually works.

If the verification fails, it does not stop. It goes back, reads the error, and tries again. The loop continues until the work is done. Shaping that loop deliberately - the step budget, the error message fed back, and the exit conditions - is Loop Engineering.

One more thing to notice: we are part of this loop too. We can interrupt Claude Code at any point, correct it, or give it more information. It works on its own, but it always listens to us.

Now that we have understood the loop, it's time to learn about the tools that power each phase.

The tools of Claude Code

A **tool** is one small ability that Claude Code gives to the model. The model cannot touch our computer directly. It can only request a tool, the tool does the work, and the result goes back to the model. Then, the model decides the next step based on that result. This request-and-respond mechanism, where the model asks for a tool and the result comes back, is called function calling. We have a detailed blog on how Function Calling works in LLMs that explains it in depth.

Claude Code has many tools, but six of them do most of the work. Let me tabulate them with a real-world comparison for your better understanding:

Tool	What it does	Like a developer...
`Read`	Reads a file and returns its content	Opening a file to read the code
`Grep`	Searches for a word or pattern inside all files	Using "Find in all files" in an editor
`Glob`	Finds files whose names match a pattern	Looking for a file by its name
`Edit`	Changes a specific part of a file	Editing a few lines of code
`Write`	Creates a new file or replaces a file	Creating a new file in the project
`Bash`	Runs any terminal command	Running the tests, building the app

Here, `Grep` and `Glob` can look strange, so let me explain them in simple words.

`Grep` is a text search. If we ask it to search for the word `loginUser`, it looks inside every file of the project and tells us which files contain that word and on which lines. It is like pressing Ctrl+F, but for the entire project at once.

`Glob` is a filename search. A pattern like `*.js` means "all files whose names end with .js". So, `Glob` answers questions like "which test files exist in this project?" without opening a single file.

And, `Bash` is the most powerful tool. It can run any command we could type in the terminal ourselves: running tests, installing packages, using git, starting a server, and etc. This one tool is what lets Claude Code verify its own work.

The best way to learn how these tools work together is by taking an example.

Example: Claude Code fixing a bug

Let's say one test in our project is failing. We open the terminal in our project folder and type as below:

```
claude
> Fix the failing login test
```

Here, the first line starts Claude Code, and the second line is our request in plain English. Notice that we did not tell it which file has the bug. We do not even know that ourselves.

Now, let's watch what Claude Code does, step by step.

Step 1: It uses `Bash` to run the test command of the project, for example `npm test`. The output says one test failed: "loginUser should return a token".

Step 2: It uses `Grep` to search for `loginUser` across the project. The search finds the function inside a file called `auth.js`.

Step 3: It uses `Read` to open `auth.js`. Now, the model can see the actual code and understand the bug. Let say the code forgets to return a value in one case.

Step 4: It uses `Edit` to fix those lines in `auth.js`.

Step 5: It uses `Bash` to run `npm test` again. All tests pass.

The whole journey looks like below:

```
Our request: "Fix the failing login test"
                          |
                          v
+-----------------------------------------------+
| Bash: run the tests      --> one test failed      |
| Grep: search "loginUser" --> found in auth.js     |
| Read: auth.js            --> understands the code |
| Edit: auth.js            --> fixes the bug        |
| Bash: run the tests      --> all tests passed     |
+-----------------------------------------------+
                          |
                          v
                    Task complete
```

Here, we can see that every tool result feeds the next decision. The failing test told it what to search. The search told it what to read. The reading told it what to edit. And, the final test run proved that the fix works.

It works perfectly.

This is the agent loop in action: gather context with `Bash`, `Grep`, and `Read`, take action with `Edit`, and verify results with `Bash` again.

How does Claude Code search a big project?

Now, the question is: why does Claude Code search first instead of reading everything?

The answer is the **context window**. The context window is the working memory of the AI model. It can hold only a limited amount of text at a time. Everything the model knows about our conversation, our files, and our commands must fit inside it.

Suppose the context window is a whiteboard of a fixed size. Every file we read gets written on the whiteboard. When the whiteboard is full, old things must be erased to make room for new things.

A real project can have thousands of files. If Claude Code tried to read all of them, the whiteboard would overflow immediately. So, it does what a smart developer does: it searches first and reads only what matters.

Deciding what goes on this whiteboard and what stays off it is called context engineering. We have a detailed blog on Context Engineering that covers this end to end.

This is why `Grep` and `Glob` come into the picture. They are simple, fast text searches. They scan thousands of files in a moment and return only a small list of matching files and lines. That small list takes very less space on the whiteboard. Then, Claude Code reads only the two or three files that actually matter.

The search works like a funnel, like below:

```
+-----------------------------+
| Thousands of files          |
+-----------------------------+
               |
               |  Grep and Glob search them
               v
+-----------------------------+
| A few matching files        |
+-----------------------------+
               |
               |  Read opens only these
               v
+-----------------------------+
| Only the code that matters  |
| is on the whiteboard        |
+-----------------------------+
```

Here, we can see that the project gets narrowed down at every step. Thousands of files become a few files, and a few files become a small amount of code on the whiteboard. The whiteboard never overflows.

There is no magic database and no special index of our code. Claude Code finds its way through a project using the same simple text searches that developers have used for decades. The intelligence is not in the search tool. The intelligence is in the model deciding what to search for.

How does Claude Code verify its own work?

I personally believe that this is the most important part of Claude Code.

The biggest weakness of a normal AI chatbot is that it cannot check its own answer. It writes code that looks correct, and looking correct is not the same as being correct.

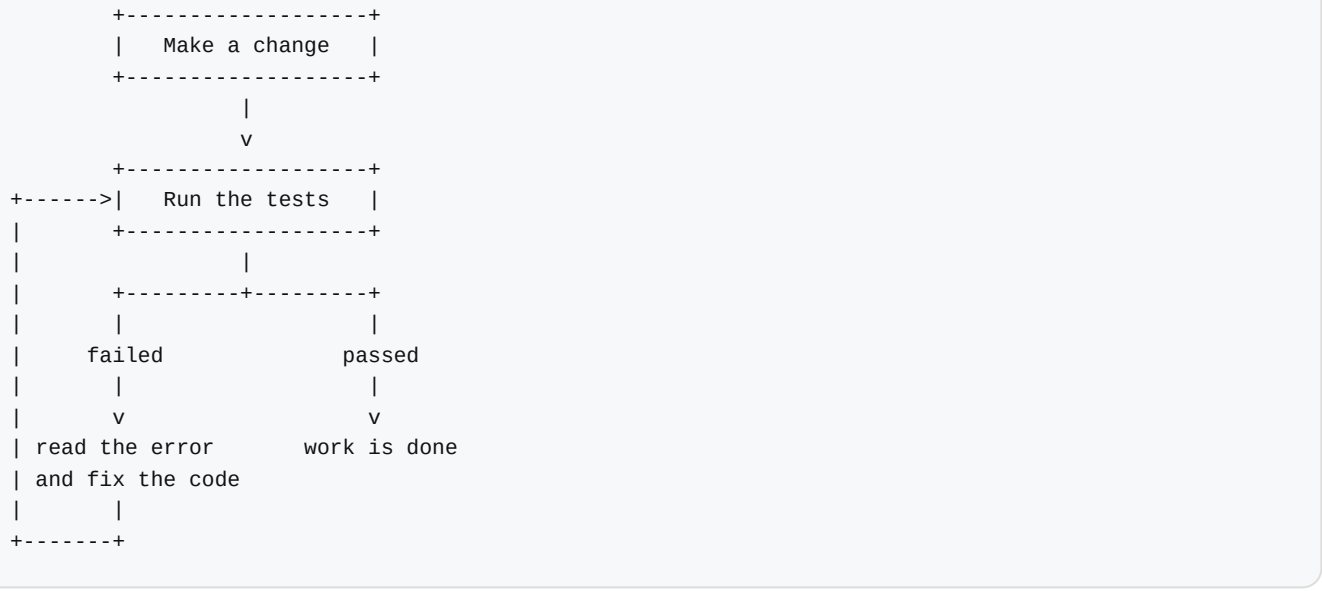
Claude Code closes this gap by running the code. After making a change, it can:

- Run the tests of the project and read the results.
- Run the compiler or build and read the errors.
- Run the program itself and check the output.

If anything fails, the error message goes right back into the loop. The model reads the error, understands what went wrong, and fixes it. This can repeat many times without us doing anything.

This pattern, where an agent inspects its own output and corrects itself, is called reflection. We have a detailed blog on the Reflection Agent that explains this step by step.

The verification loop looks like below:



Here, we can see that a failed test does not end the work. The error goes back into the loop, the code gets fixed, and the tests run again. The loop only ends when the tests pass.

This is exactly how a good developer works. We never say "the bug is fixed" without running the code. Claude Code follows the same discipline.

And, we can help it here. If we give Claude Code something to verify against, it performs much better. For example, we can say: "Write a failing test first, then fix the code, then run the tests." Now, it has a clear target, and it will keep looping until the target is hit.

CLAUDE.md: The project memory

Every new Claude Code session starts with a fresh, empty context window. Remember the whiteboard? Each session begins with a clean whiteboard.

But, here is the catch. Our project has rules that never change: which language we use, how to run the tests, which folders must not be touched. Explaining these rules again in every session would be painful.

So, here comes **CLAUDE.md** into the picture.

CLAUDE.md is a simple text file that we keep in our project folder. Claude Code reads it automatically at the start of every session. Whatever we write in this file becomes standing instructions for Claude

Code.

A small CLAUDE.md can look like below:

```
# Project Notes
- We use Kotlin for all new code.
- Run the tests with: ./gradlew test
- Do not edit files inside the generated/ folder.
```

Here, we have written the language of the project, the exact test command, and one folder that must never be touched. Now, Claude Code knows all of this in every session without us repeating it.

Suppose a new developer joins our team and we hand them a one-page note: "Here is how our team works." CLAUDE.md is exactly that note, written for Claude Code. When one of those rules grows into a full procedure with its own steps and files, we keep it out of this always-loaded note and write it as an Agent Skill instead, which Claude Code loads only when a task actually needs it.

Permissions: How we stay in control

Now, the next big question is: an agent that can edit files and run commands is powerful, so how do we stay safe?

The answer is **permissions**. By default, Claude Code asks for our approval before it edits a file or runs a command that changes something. We see a prompt like below:

```
Claude wants to run: npm test
Do you want to allow this?
> 1. Yes
  2. Yes, and do not ask again for this command
  3. No, and tell Claude what to do differently
```

Here, this is a simplified version of the prompt, just for the sake of understanding. Nothing changes on our computer until we say yes. And, if we trust a command like `npm test`, we can allow it permanently so Claude Code never asks for it again.

Claude Code also has different **permission modes** that we can switch between:

- **Manual:** Claude Code asks before every file edit and every command. This is the default.
- **Accept edits:** File edits are approved automatically, but other commands still need our approval. We use this when we already trust the direction of the work.
- **Plan:** Claude Code can only read and think. It cannot edit anything. We will learn about this mode in the next section.
- **Auto:** Claude Code works without asking, with safety checks running in the background. This is meant for automated setups where no human is sitting at the keyboard.

One more safety net exists: **checkpoints**. Before Claude Code edits any file, it saves a snapshot of that file. If we do not like the result, we can rewind everything back to how it was. Every file edit is reversible.

Plan mode, subagents, and hooks

Till now, we have learned the core of Claude Code. Now, let's learn three features that make it even more useful.

Plan mode

In **plan mode**, Claude Code is read-only. It explores the project, thinks, and then presents a plan: "Here is what I will change and why." It does not touch a single file.

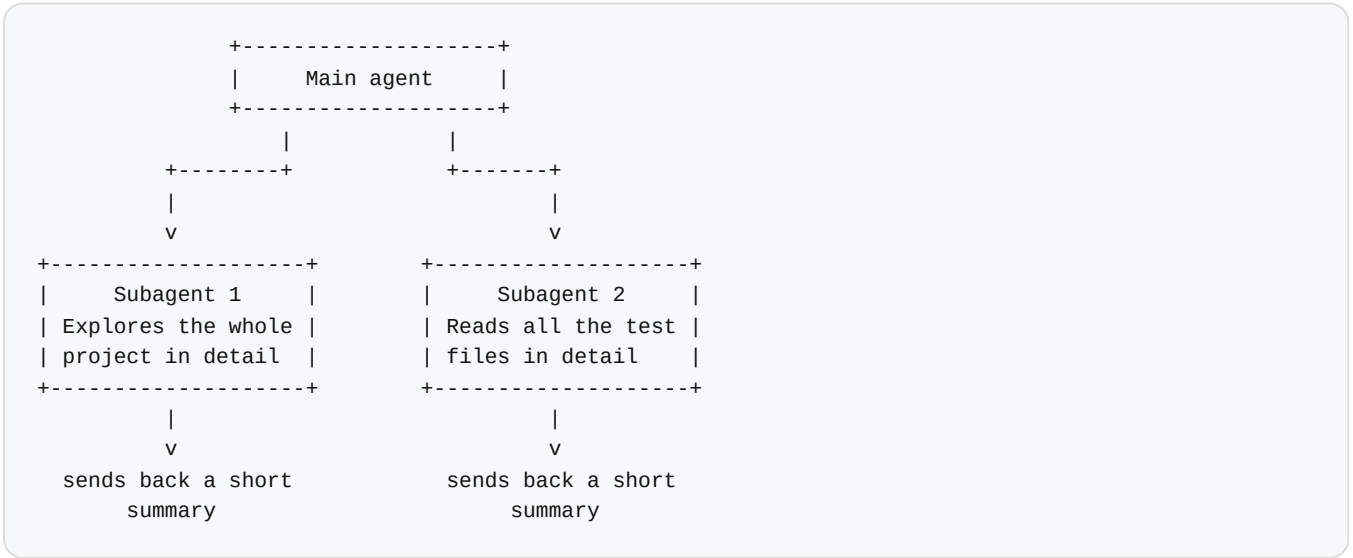
We read the plan, correct it through conversation, and only then allow the implementation. It is like agreeing on the blueprint of a house before any construction starts. For big changes, this two-phase approach gives much better results than jumping straight to code.

Subagents

Remember the whiteboard problem? Long tasks fill the context window quickly. A big exploration alone can fill the entire whiteboard.

So, here comes the **subagent** to the rescue.

A subagent is a helper agent with its own fresh context window, which means its own separate whiteboard. The main agent gives it one focused task, the subagent does all the messy exploration on its own whiteboard, and then it returns only a short summary.



Here, the two subagents read a huge amount of code, but the main agent receives only two short summaries. The main whiteboard stays clean, and the subagents can even work at the same time.

Subagents are one way to keep the whiteboard clean. Another is to summarize the older history in place so it still fits - we have a detailed blog on how context compaction works that covers this end to end.

Hooks

A **hook** is a small script that runs automatically when a specific event happens inside Claude Code. For example: after every file edit, run the code formatter.

The important difference is this: an instruction in CLAUDE.md is a request that the model follows, but a hook is a rule that the system enforces. A hook runs every single time, without exception. We use hooks when something must always happen.

Putting it all together

Let's recap the complete picture.

Claude Code is a brain plus a body. The brain is the Claude model, which reads, reasons, and decides. The body is the set of tools: `Read`, `Grep`, `Glob`, `Edit`, `Write`, and `Bash`. The agent loop connects them: gather context, take action, verify results, and repeat until done.

CLAUDE.md gives it memory about our project. Permissions and checkpoints keep us in control. Plan mode separates thinking from doing. Subagents keep the working memory clean. Hooks enforce the rules that must never be skipped.

Now we must have understood how Claude Code works. The next time we watch it fix a bug from a single English sentence, we know there is no magic inside. There is a simple loop, the same loop every good developer follows: understand the code, change the code, and prove that the change works.

Frequently Asked Questions

What is the difference between Claude Code and a normal AI chatbot?

A normal AI chatbot can only produce text, while Claude Code can act on our project. The chatbot cannot see our project, cannot make changes, and cannot verify its answer. Claude Code gives the model tools to read files, edit code, run commands, and check its own work.

Does Claude Code build a special index of our codebase?

No. There is no magic database and no special index of our code. Claude Code uses simple text searches like `Grep` and `Glob` to find the few files that matter. The intelligence is not in the search tool but in the model deciding what to search for.

Can Claude Code change files without our approval?

Not by default. Claude Code asks for our approval before it edits a file or runs a command that changes something. We can switch modes, for example to approve file edits automatically. Before any file edit, it also saves a checkpoint, so every edit is reversible.

What is the difference between a CLAUDE.md instruction and a hook?

An instruction in CLAUDE.md is a request that the model follows, while a hook is a rule that the system enforces. A hook is a small script that runs every single time a specific event happens, without exception. We use hooks when something must always happen.

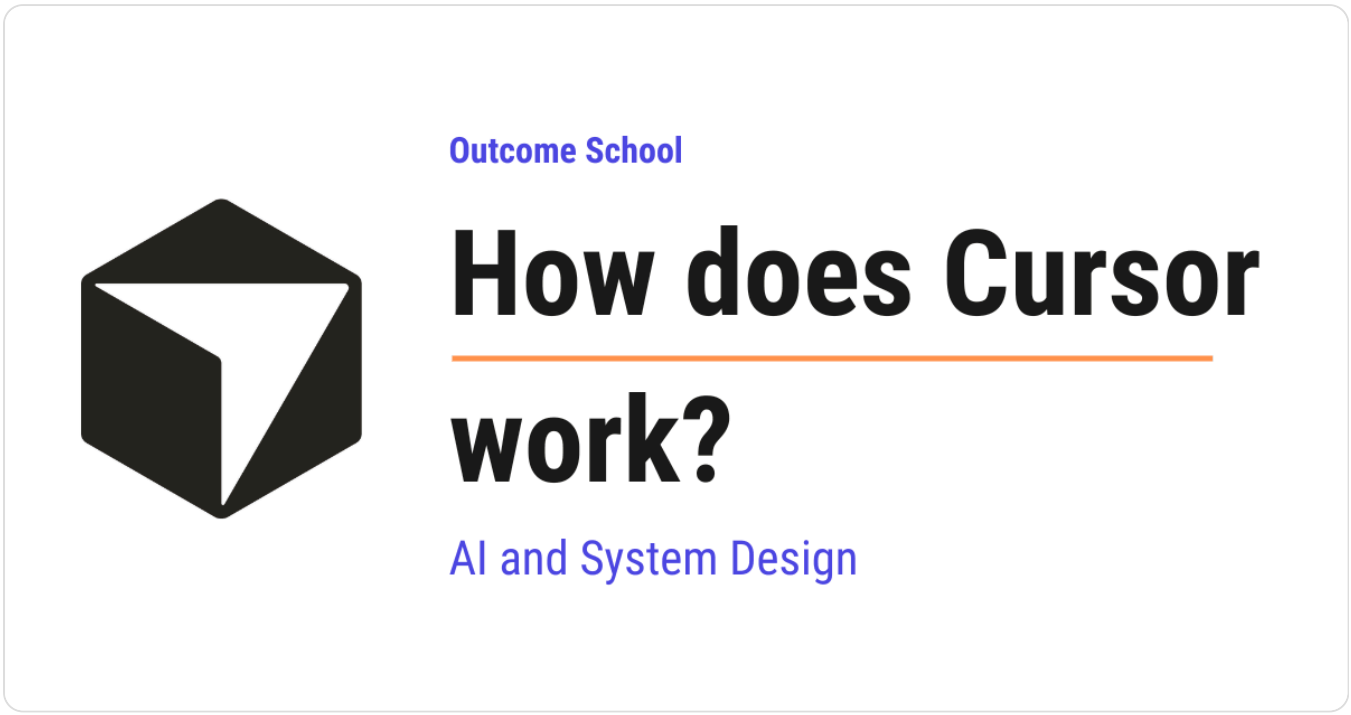
When should we use plan mode in Claude Code?

We should use plan mode for big changes. In plan mode, Claude Code is read-only. It explores the project and presents a plan without touching a single file. We correct the plan through conversation and only then allow the implementation, which gives much better results than jumping straight to code.

LESSON 11.8

How does Cursor work?

Lesson 11.8 · ~17 min · Published July 22, 2026 · https://outcomeschool.com/blog/how-does-cursor-work



Cursor is an AI code editor. It is a place where we write our code, but with an AI helper that understands our whole codebase and can suggest, explain, and write code for us.

In this blog, we will learn about how Cursor works. We will also see what Cursor is, why it is a code editor with an AI brain on top, how it indexes our codebase into embeddings and searches it by meaning, how Tab autocomplete, Chat, and Agent mode work, how Cursor applies changes through a diff, why it uses different models for different jobs, and how it tries to keep our code private.

We will cover the following:

- What is Cursor?
- Cursor = Code Editor + AI
- The big idea behind Cursor
- How does Cursor understand our code?
- How does Cursor index our codebase?
- How does Tab autocomplete work?
- How does the Chat work?
- How does the Agent mode work?
- How does Cursor apply the changes?
- Why does Cursor use different models?
- How does Cursor keep our code private?

- The complete flow of Cursor

Let's get started.

What is Cursor?

Cursor is an AI code editor. It is a place where we write our code, but with an AI helper sitting right next to us.

Let's say we are writing code. Normally, we type every single line by ourselves. We think, we type, we fix the mistakes, and we repeat. It takes time.

Now, imagine a smart friend sitting next to us. This friend has read a lot of code. When we start typing, the friend already guesses what we want to write next. When we get stuck, we simply ask the friend, and the friend writes the code for us. When we want to build a whole feature, we just describe it in plain English, and the friend does the work across many files.

That smart friend is Cursor.

So, Cursor is a code editor where an AI is always ready to help us read, write, and change our code.

Cursor = Code Editor + AI

Let's understand both parts.

Code Editor: A code editor is a simple app where we open our files and write our code. Cursor is built on top of a very popular code editor called VS Code. So, if we have ever used VS Code, then Cursor will feel the same. All the buttons, the file explorer on the left, the tabs on the top, everything feels familiar.

AI: This is the special part. Cursor connects our editor to powerful AI models. These AI models are the same kind of models that power tools like ChatGPT. They can read text, understand it, and write text. And code is also just text. So, these AI models can read our code and write new code for us.

So, Cursor takes a normal code editor and adds a powerful AI brain on top of it. That is the whole idea.

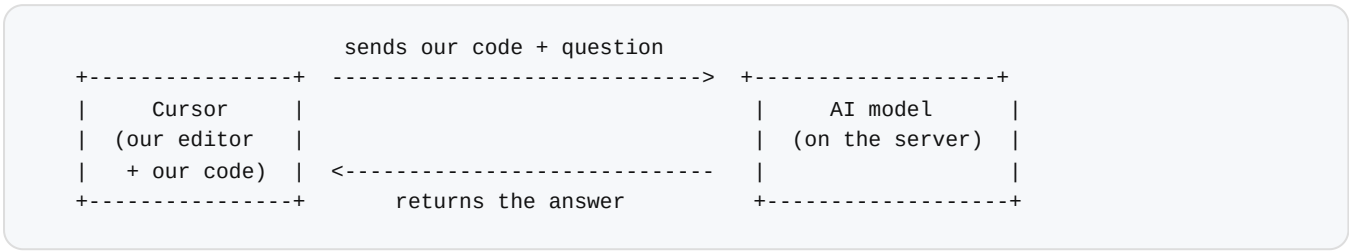
The big idea behind Cursor

Before jumping into the details, we must understand the big idea. It will make everything else easy.

The AI model does not live inside Cursor. The AI model lives on a big computer far away, called a server. This server belongs to companies that build these models.

So, Cursor is like a bridge. On one side, there is us and our code. On the other side, there is the powerful AI model on the server.

We can picture it like below:



Here, we can see that our code never runs the AI by itself. Cursor sends the needed pieces to the AI model and brings the answer back.

When we ask for help, Cursor does three things:

- **First,** it collects the right pieces of our code. This is called context.
- **Then,** it sends our question along with that context to the AI model.
- **Finally,** it takes the answer from the AI model and puts it back into our code.

Here, the most important word is context. The AI model has never seen our project before. It does not know our files. So, Cursor must send the right pieces of our code to the AI model, so that the AI can give a useful answer.

Let's say we ask a doctor for advice over the phone. The doctor is very smart, but the doctor cannot see us. So, we must describe our problem clearly. The better we describe it, the better the advice.

In the same way, the AI model is very smart, but it cannot see our project. So, Cursor must describe our code clearly to the AI model. This is the real magic of Cursor. It is very good at picking the right pieces of our code and sending them to the AI.

Now, let's understand how Cursor does this.

How does Cursor understand our code?

Let's say our project has a thousand files. When we ask a question, the AI model cannot read all thousand files at once. It is too much. It would be slow and very costly.

So, Cursor must find only the few files that matter for our question. This is the real problem to solve.

For example, we ask, "Where do we check the user's password?" Cursor must find the file that handles the password. It must not send the file that handles the app color or the file that handles the images.

So, how does Cursor find the right files? Here comes the indexing to the rescue. Let's understand it.

How does Cursor index our codebase?

Indexing means Cursor reads all our files once and prepares them in a smart way, so that later it can search through them very fast.

Let's understand this step by step.

Step 1: Cursor breaks our code into small pieces. Each piece is called a chunk. A chunk is usually a small part of a file, like one function or a few lines that belong together.

Step 2: Cursor turns each chunk into a special list of numbers. This list of numbers is called an embedding.

Now, the word embedding sounds scary, so let's understand it in simple words.

An embedding is a list of numbers that captures the meaning of a piece of text.

The idea is beautiful. Two chunks that mean similar things get similar numbers. Two chunks that mean different things get very different numbers.

Let's say we have two chunks. One chunk is about checking a user's password. Another chunk is also about logging in a user. These two are about similar things. So, their numbers will be close to each other. But a chunk about changing the app color will have very different numbers.

Step 3: Cursor stores all these numbers in a special database. This database is very good at one job. Given some numbers, it can quickly find other numbers that are close to them.

So, now our whole codebase is turned into numbers and stored, ready to be searched.

We have a detailed blog on how a Vector Database works that covers this in depth.

The whole indexing pipeline looks like below:

```
Our code files
     |
     |  break into small pieces
     v
  Chunks        (one function, or a few related lines)
     |
     |  turn each chunk into numbers
     v
Embeddings      (a list of numbers that captures the meaning)
     |
     |  store all the numbers
     v
Vector database (ready to search by meaning)
```

Now, when we ask, "Where do we check the user's password?", here is what happens:

- Cursor turns our question into numbers, the same way it turned the code chunks into numbers.
- Then, it searches the database for the code chunks whose numbers are closest to our question's numbers.

- The closest chunks are most likely the ones that matter for our question.

This way, Cursor finds the right pieces of code without reading the whole project. This search by meaning is often called semantic search.

Note: Cursor is careful here. For most plans, it does not store our actual code on its own servers. It mainly stores these numbers. We will talk more about this in the privacy section.

This is how Cursor understands our codebase. Now, let's move to the features we use every day.

How does Tab autocomplete work?

The first feature we notice in Cursor is Tab. When we type code, Cursor shows a grey suggestion for the next lines. We press the Tab key, and the suggestion becomes real code.

But this is not a normal autocomplete. A normal editor only completes one word. Cursor predicts the next full edit. It can suggest many lines at once. It can even suggest changes a few lines away from where we are typing.

Let's understand how it works.

Cursor uses a special AI model that is trained for one job only, to predict the next edit. When we type, Cursor sends the model a few things:

- The code around our cursor, both above and below.
- The recent changes we just made.
- Some related pieces from the rest of our project.

The model looks at all this and guesses what we are most likely to write next. Then, it shows that guess as a grey suggestion.

Here, the important point is speed. This suggestion must appear in a fraction of a second. If it is slow, it becomes annoying. So, Cursor uses a smaller and faster model for Tab, not the big heavy model. This smaller model is quick, and it is trained to be very good at this one task.

Let's say we just wrote a line that creates a variable for the user's name. The model sees this. It guesses that our next line will probably use that name to greet the user. So, it suggests that line for us. We press Tab, and it is done.

This is how the Tab autocomplete makes our life easier. It removes the boring typing and lets us move fast.

How does the Chat work?

The next feature is Chat. On the side of the editor, there is a chat box. We can ask any question about our code, and we get an answer.

For example, we can ask, "What does this function do?" or "Why is this test failing?" We can also ask it to write new code for us.

Here is how the Chat works, step by step.

Step 1: We type our question in the chat box.

Step 2: Cursor gathers the context. It uses the indexing we learned about above to find the code chunks that matter for our question. It also adds the file we are currently looking at. We can also point to specific files ourselves.

Step 3: Cursor builds a full message for the AI model. This message contains our question and the gathered context. This step of building a good message is called the prompt.

Step 4: Cursor sends this prompt to the big AI model on the server.

Step 5: The AI model reads everything and writes an answer. The answer comes back to Cursor and appears in the chat.

So, the Chat is just a smart conversation. But the reason it feels smart is the context. Because Cursor sends the right pieces of our code, the AI can answer about our actual project and not just talk in general.

This is how the Chat works.

How does the Agent mode work?

Now, it's time to learn about the most powerful feature of Cursor, the Agent mode.

In the Chat, the AI only talks to us. In the Agent mode, the AI can do the work by itself. It can read files, write files, and even run commands. It keeps working step by step until the whole task is done.

Let's understand it with an example. We tell the Agent, "Add a dark mode to my app."

A normal chat would just tell us what to do. But the Agent actually does it. Here is how.

The AI model is given a set of tools. A tool is simply an action that the AI is allowed to take. For example:

- A tool to read a file.
- A tool to search the codebase.
- A tool to make an edit in a file.
- A tool to run a command in the terminal.

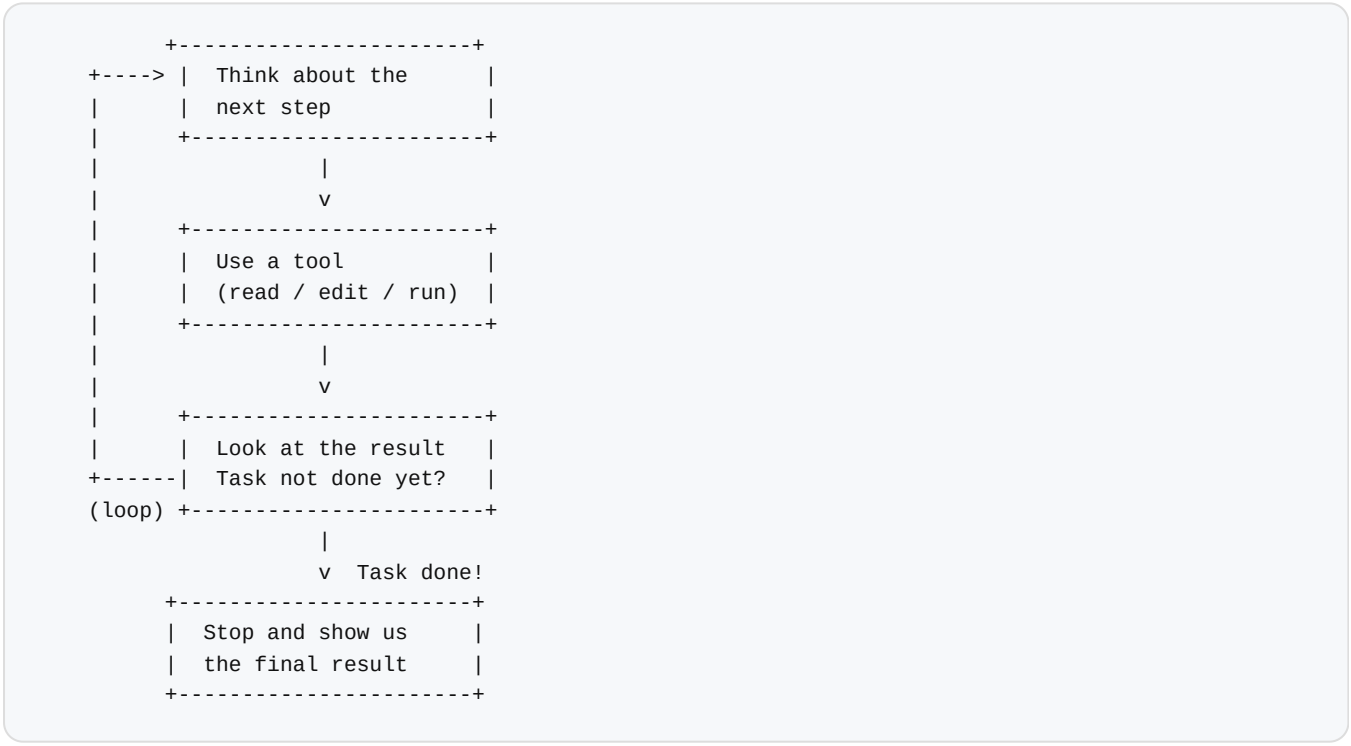
Cursor can also reach tools beyond these built-in ones, like a database or an issue tracker, through an open standard called MCP. We have a detailed blog on What is MCP (Model Context Protocol)? that explains this step by step.

Now, the Agent works in a loop. Let's see the loop.

- **First,** the AI thinks about the task and decides the next step. Let's say it decides to first read the settings file.
- **Then,** it uses the read tool to open that file. Cursor runs the tool and gives the file content back to the AI.
- **After that,** the AI reads the content and decides the next step. Now, it decides to edit the file to add the dark mode option.
- **Then,** it uses the edit tool to make the change.
- **After that,** it may run the app to check if the change works.

This loop keeps going. Think, act, look at the result, think again. The AI repeats this until the task is complete. This think-act-repeat loop is what makes it an agent. Designing that loop well - how many turns it gets, what it sees each turn, and when it must stop - is Loop Engineering.

We can picture the loop like below:



Here, we can see that the AI does not do everything in one shot. It works one small step at a time, and it checks the result after every step.

Here, we are still in control. Cursor can ask us for permission before it does something risky, like running a command. We can review every change before we accept it.

So, the Agent mode turns Cursor from a helper that talks into a helper that works. This is the beauty of Cursor.

How does Cursor apply the changes?

Now, the next question is: when the AI writes new code, how does it go into our files?

Let's say the AI wants to change ten lines in a big file of five hundred lines. Sending the whole new file every time would be slow and wasteful.

So, Cursor is smart here. The big AI model does not rewrite the whole file. It only describes the change. It says something like, "In this function, replace these lines with these new lines."

Then, Cursor uses another small and fast model, called the Apply model. The only job of this model is to take that described change and merge it neatly into our real file.

After that, Cursor shows us the change as a diff. A diff is a simple view that shows the old lines in red and the new lines in green. So, we can clearly see exactly what will change.

We look at the diff. If we like it, we click Accept, and the change becomes real. If we do not like it, we click Reject, and nothing changes.

This is how Cursor applies the changes. We always stay in control, because nothing changes until we accept it.

Why does Cursor use different models?

By now, we have noticed something. Cursor does not use only one AI model. It uses different models for different jobs. This is a smart choice.

The reason is simple. A big and heavy model is very smart, but it is slow and costly. A small and fast model is quick, but it is not as smart. So, Cursor uses the right model for the right job.

Let me tabulate these models for your better understanding so that the whole picture becomes clear.

Job	Model used	Why
Tab autocomplete	A small, fast model	The suggestion must appear instantly while we type.
Chat and Agent	A big, smart model	The hard thinking and code writing need the smartest model.
Applying the change	A small, fast Apply model	Merging a described change into a file is a simple, quick task.

Here, we can see that each model has one clear job. The big model does the deep thinking. The small models handle the quick tasks. This mix makes Cursor both smart and fast.

How does Cursor keep our code private?

Our code is important, so a natural question arises. Is our code safe?

Cursor has a setting called Privacy Mode. When Privacy Mode is on, our code is not stored by Cursor after our request is done. The code is used only to answer our question, and then it is not kept.

Here, we must understand one thing clearly. To answer our question, Cursor must send some pieces of our code to the AI model on the server. This is needed, because the AI lives on the server, not on our computer. But with Privacy Mode, that code is not saved for later.

Also, as we learned, the codebase index mostly stores numbers, the embeddings, and not our full code text. This adds another layer of safety.

So, Cursor gives us the power of AI, while still trying to protect our code.

The complete flow of Cursor

Now that we have learned about all the parts, let's put the whole flow together in simple steps.

- **First,** Cursor reads our project once and turns it into numbers, the embeddings, and stores them. This is indexing.
- **Then,** when we type, the fast Tab model predicts our next edit and shows a grey suggestion.
- **After that,** when we ask a question in Chat, Cursor searches the index, finds the right code chunks, and builds a prompt.
- **Then,** it sends the prompt to the big AI model on the server, and shows us the answer.
- **In Agent mode,** the AI uses tools to read, edit, and run code in a loop until the task is done.
- **Finally,** the Apply model merges the AI's changes into our files, and we accept or reject each change through a diff.

So, Cursor is a bridge between our code and a powerful AI model. It is very good at picking the right pieces of our code, sending them to the AI, and putting the answer back into our files.

This is the beauty of Cursor. It makes our life easy by removing the boring work, while keeping us fully in control of every change.

Now, we must have understood how Cursor works.

Frequently Asked Questions

Is Cursor different from VS Code?

Cursor is built on top of VS Code, so it feels the same. The file explorer, the tabs, and the buttons all feel familiar. The difference is that Cursor connects the editor to powerful AI models, which can read our code, answer questions about it, and write new code for us.

Does the AI model run on our own computer in Cursor?

No. The AI model lives on a server, not inside Cursor. Cursor works like a bridge. It collects the right pieces of our code, sends them with our question to the AI model, and puts the answer back into our code.

Does Cursor send our whole project to the AI model?

No. Sending a thousand files would be slow and very costly. Cursor indexes our codebase into embeddings and searches it by meaning, so it can find only the few code chunks that matter for our question. It then sends just those pieces as context.

What is the difference between Chat and Agent mode in Cursor?

In Chat, the AI only talks to us and answers our questions. In Agent mode, the AI does the work by itself. It uses tools to read files, search the codebase, edit files, and run commands in a loop until the whole task is done.

Can Cursor change our code without our approval?

No. Cursor shows every change as a diff, with old lines in red and new lines in green. If we like it, we click Accept, and if not, we click Reject and nothing changes. In Agent mode, Cursor can also ask for permission before risky actions like running a command.

Personal offline study copy · Content © Outcome School / Amit Shekhar